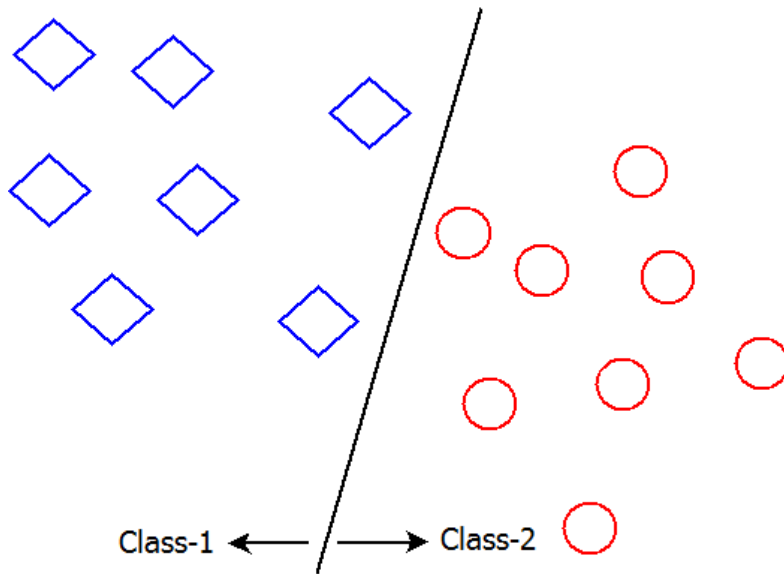


מחט בערימת דאטה:

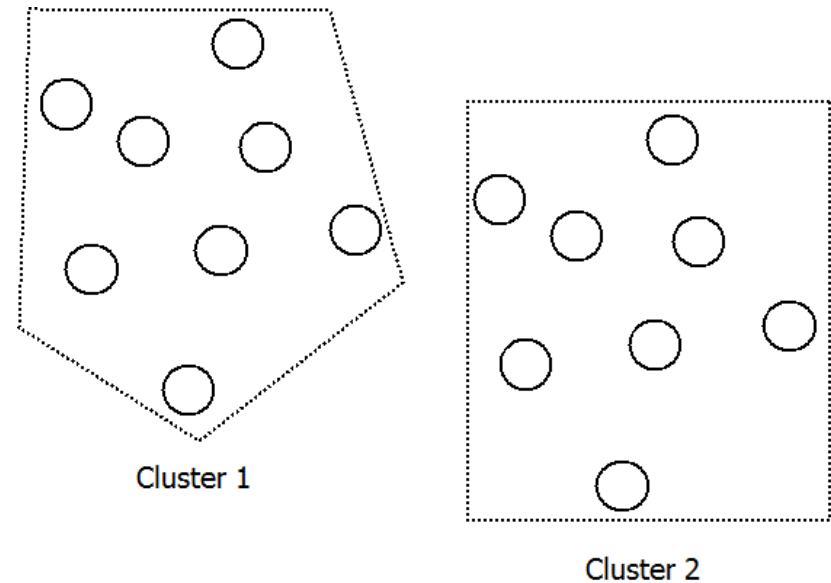
CLUSTERING

CLASSIFICATION VS. CLUSTERING

SUPERVISION AVAILABLE



UNSUPERVISED



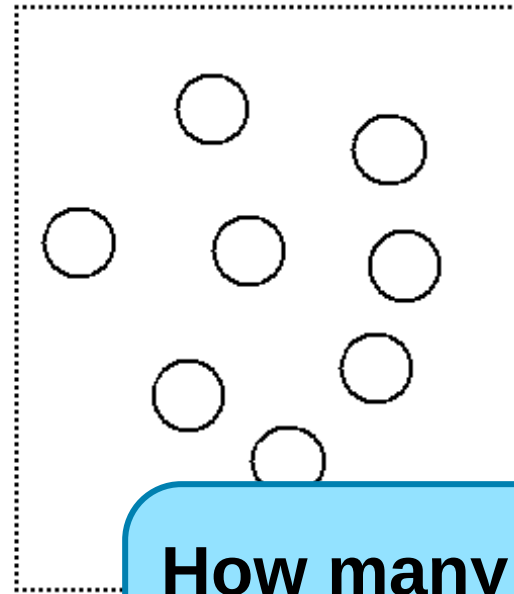
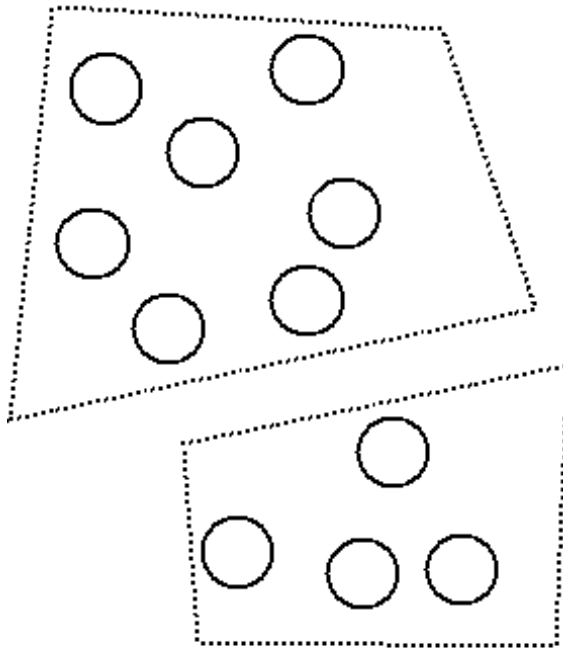
Unsupervised learning:
learning from raw
(unlabeled) data

Unsupervised Learning

- Supervised learning used labeled data pairs (x, y) to learn a function $f : X \rightarrow Y$.
- But, what if we don't have labels?
- No labels = **unsupervised learning**
 - Only some points are labeled = **semi-supervised learning**

Clustering

- **Clustering** is the unsupervised grouping of data points. Very useful for **exploration**.



**How many clusters?
How to identify
them?**

High Dimensional Data

- Given a cloud of data points we want to understand its **structure**



Clustering

- Given a **set of points**, with a notion of **distance** between points, **group the points** into some number of *clusters*, so that
 - Members of a cluster are close/similar to each other
 - Members of different clusters are dissimilar
- Usually:
 - Points are in a high-dimensional space
 - Similarity is defined using a distance measure

Clustering: Example Problems

- ?

Example Problem: Documents

Finding topics:

- Represent a document by a vector $(x_1, x_2, \dots, x_k)$, where $x_i = 1$ iff the i^{th} word (in some order) appears in the document
- **Documents with similar sets of words may be about the same topic**

Example Problem: Galaxies

- A catalog of 2 billion “sky objects” represents objects by their radiation in 7 dimensions (frequency bands)
- **Problem:** Cluster into similar objects, e.g., galaxies, nearby stars, quasars, etc.



Example Problem: Music CDs

- Music is divided into **categories**, and customers prefer a few categories
 - But what are categories really?
- Represent a CD by a set of customers who bought it
- Similar CDs have similar sets of customers, and vice-versa

Example Problem: Music CDs

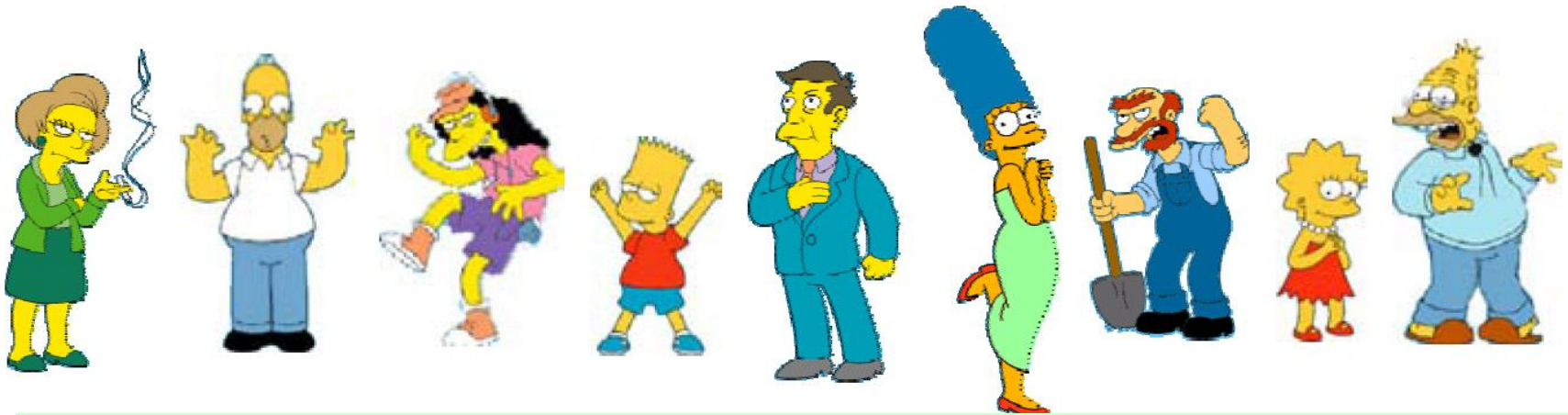
Space of all CDs:

- Think of a space with one dim. for each customer
 - Values in a dimension may be 0 or 1 only
 - A CD is a point in this space $(x_1, x_2, \dots, x_k)$,
where $x_i = 1$ iff the i^{th} customer bought the CD
- For Amazon, the dimension is tens of millions
- **Task:** Find clusters of similar CDs

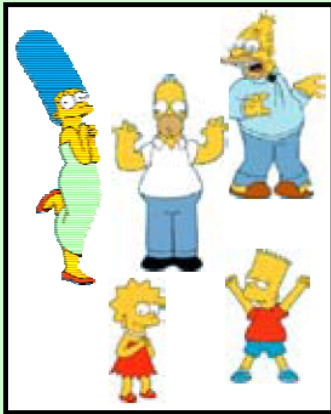
Clustering is a Hard Problem!

- Many ways to cluster
 - Clustering is in the eye of the beholder

WHAT IS A NATURAL GROUPING AMONG THESE OBJECTS?



Clustering is subjective



Simpson's Family



School Employees



Females

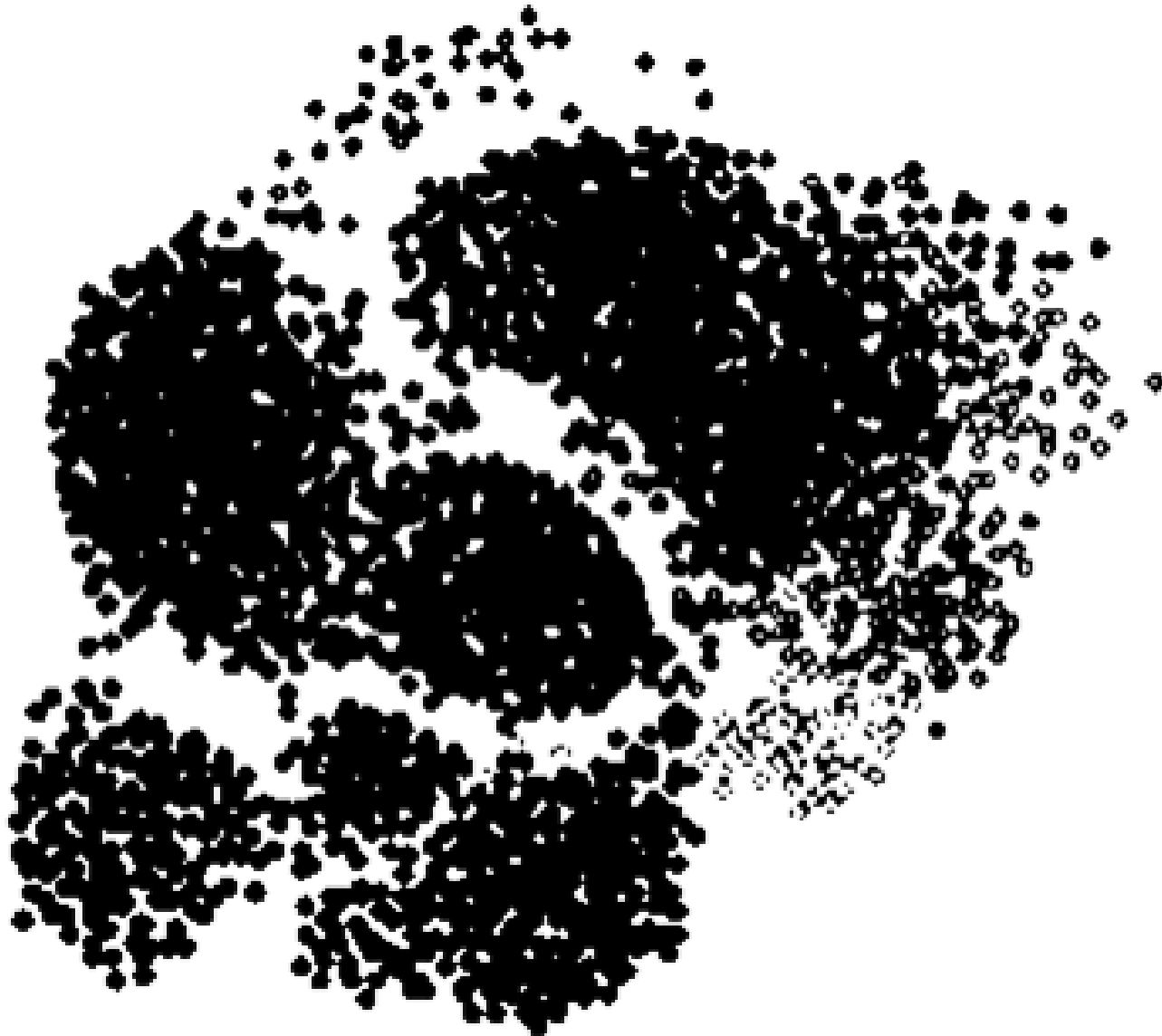


Males

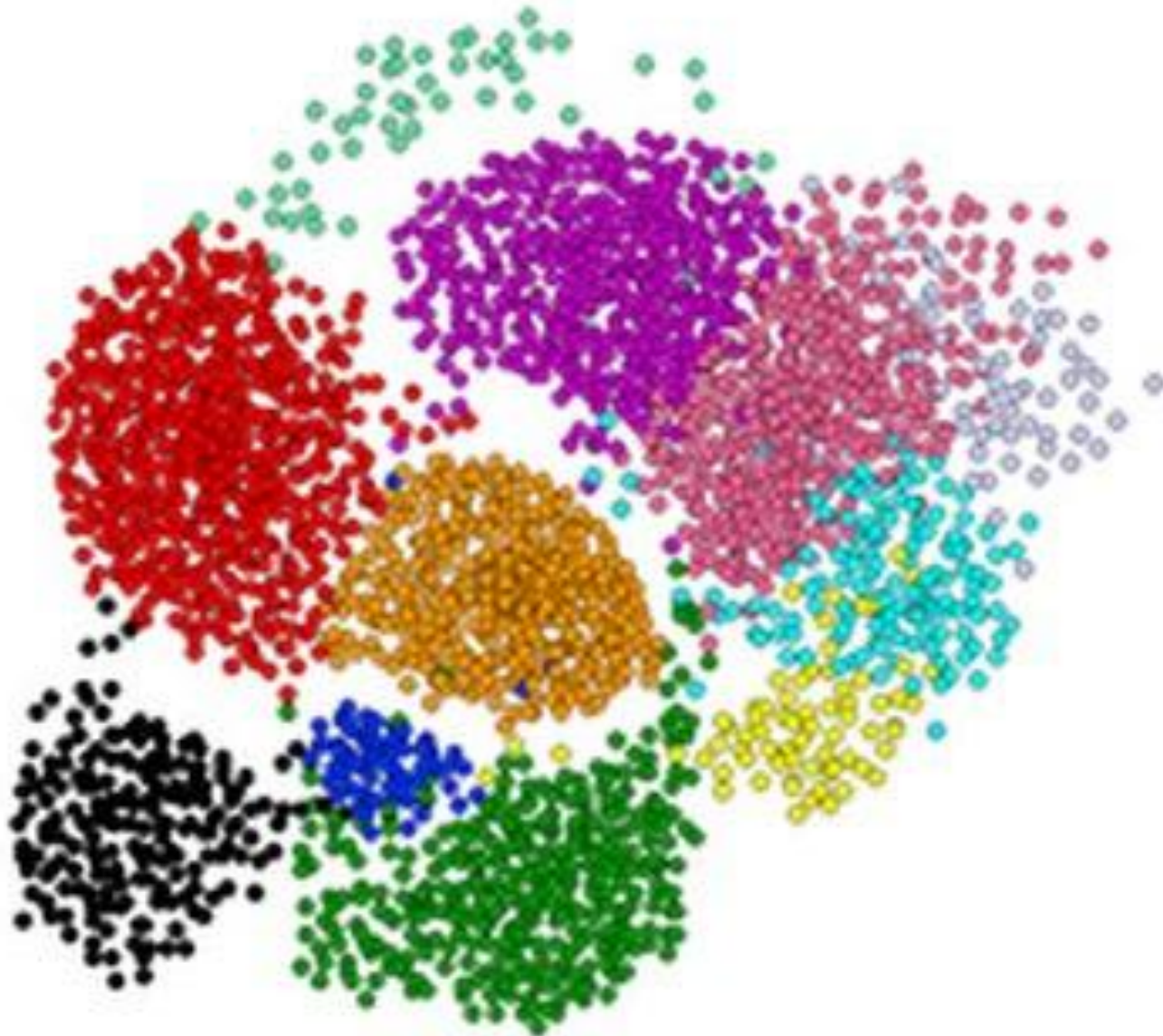
Clustering is a Hard Problem!

- Many ways to cluster
 - Clustering is in the eye of the beholder
- Clustering in two dimensions looks easy
- Clustering small amounts of data looks easy
- And in most cases, looks are not deceiving
- Many applications involve 10 or 10,000 dimensions
- **High-dimensional spaces look different:** Almost all pairs of points are at about the same distance

Clustering is a Hard Problem!



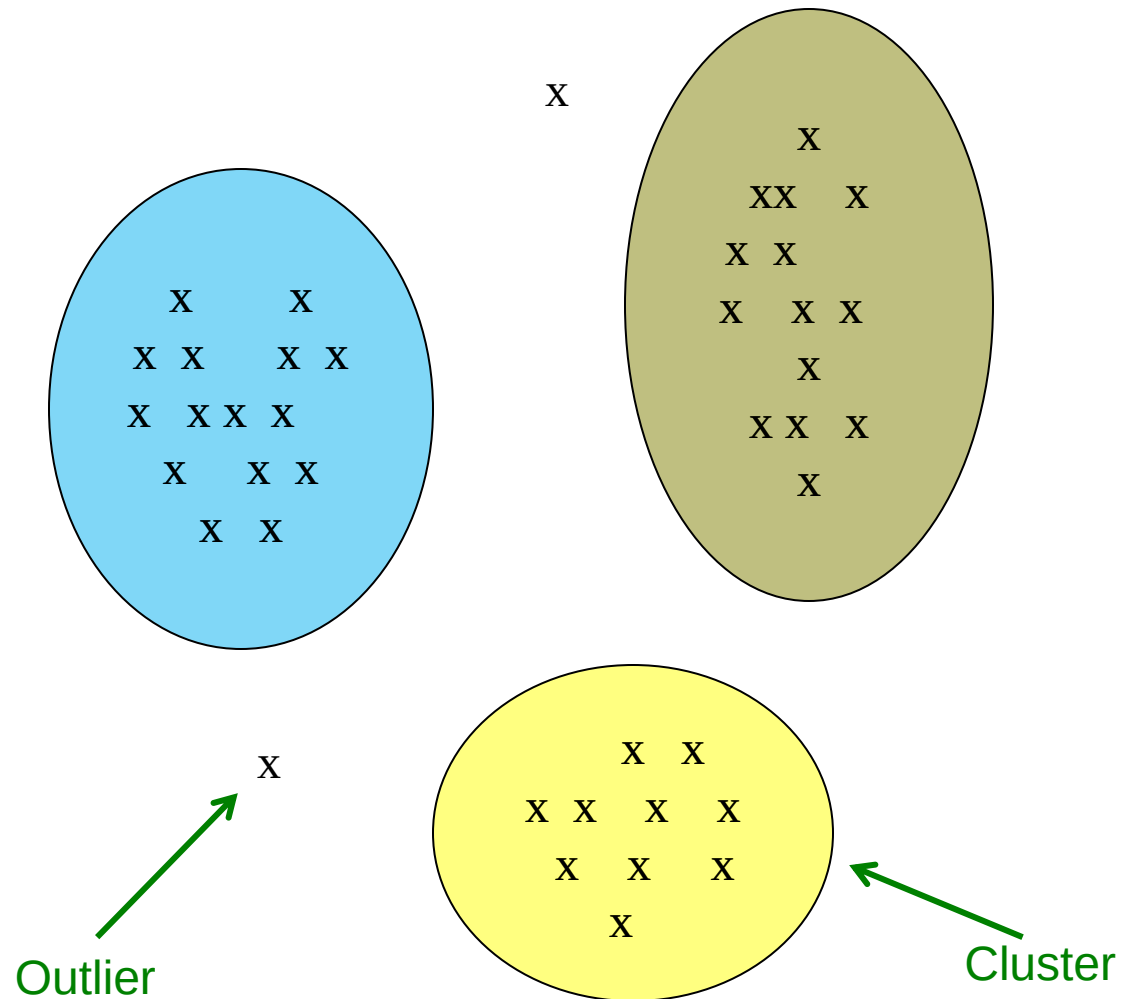
Clustering is a Hard Problem!



Types of Clustering

- Hard vs. soft
 - Each point belongs to exactly one cluster, or has a set of membership levels?
- Outliers or no outliers
- Today: hard, no outliers

OUTLIERS



Clustering

- Given a **set of points**, with a notion of **distance** between points, **group the points** into some number of *clusters*, so that
 - Members of a cluster are close/similar to each other
 - Members of different clusters are dissimilar
- Usually:
 - Points are in a high-dimensional space
 - Similarity is defined using a distance measure

Similarity Measures

What is Similarity?



**Hard to
define! But we
*know it when
we see it***

- The real meaning of similarity is a philosophical question.

Intuitions behind desirable distance measure properties

- $D(A,B) = D(B,A)$ *Symmetry*
- $D(A,A) = 0$ *Constancy of Self-Similarity*
- $D(A,B) = 0$ If $A = B$ *Identity of indiscernibles*
- $D(A,B) \leq D(A,C) + D(B,C)$ *Triangular Inequality*

Intuitions behind desirable distance measure properties

- $D(A,B) = D(B,A)$ *Symmetry*
 - *Otherwise you could claim "Alex looks like Bob, but Bob looks nothing like Alex"*
- $D(A,A) = 0$ *Constancy of Self-Similarity*
 - *Otherwise you could claim "Alex looks more like Bob, than Bob does"*
- $D(A,B) = 0$ iff $A = B$ *Identity of indiscernibles*
 - *Otherwise there are objects in your world that are different, but you cannot tell apart.*
- $D(A,B) \leq D(A,C) + D(B,C)$ *Triangular Inequality*
 - *Otherwise you could claim "Alex is very like Bob, and Alex is very like Carl, but Bob is very unlike Carl"*

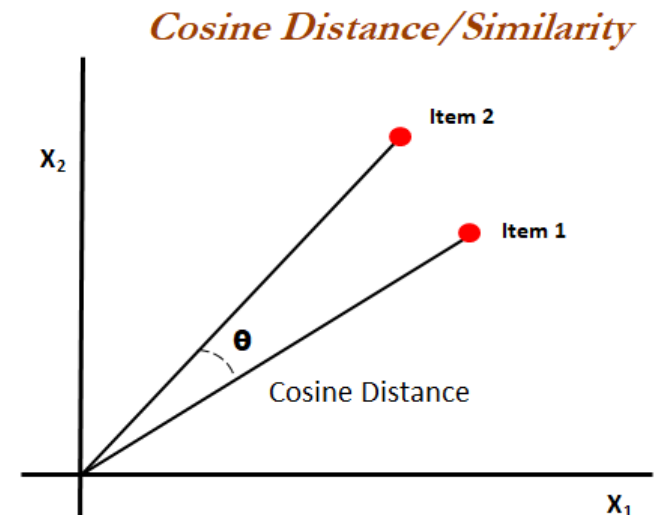
Cosine, Jaccard, Euclidean

- We have a choice when we pick distance metrics:

–**Vectors**: Measure similarity by the **cosine distance (=1- cosine similarity)**

- Measures the angle
- Magnitude ignored

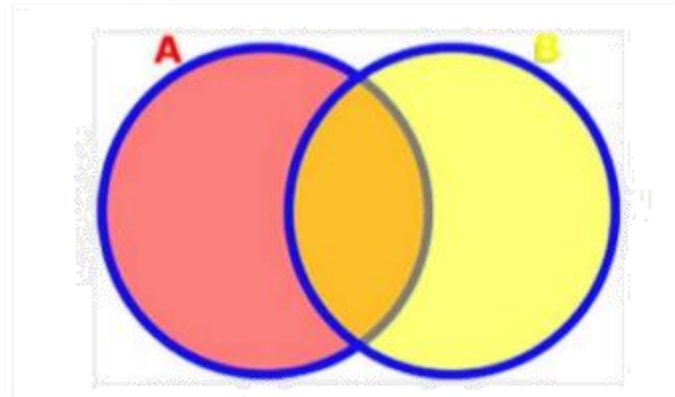
$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}},$$



Cosine, Jaccard, Euclidean

- We have a choice when we pick distance metrics:
 - **Sets:** Measure similarity by the **Jaccard distance**

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$



Cosine, Jaccard, Euclidean

- We have a choice when we pick distance metrics:
 - **Points:** Measure similarity by **Euclidean distance**

Distance Measures: Minkowski Metric

- Suppose two object x and y both have p features

$$x = (x_1, x_2, \dots, x_p)$$

$$y = (y_1, y_2, \dots, y_p)$$

- The Minkowski metric is defined by

$$d(x, y) = \sqrt[r]{\sum_{i=1}^p |x_i - y_i|^r}$$

- Most Common Minkowski Metrics

$r = 2$ (Euclidean distance)

$$d(x, y) = \sqrt{\sum_{i=1}^p |x_i - y_i|^2}$$

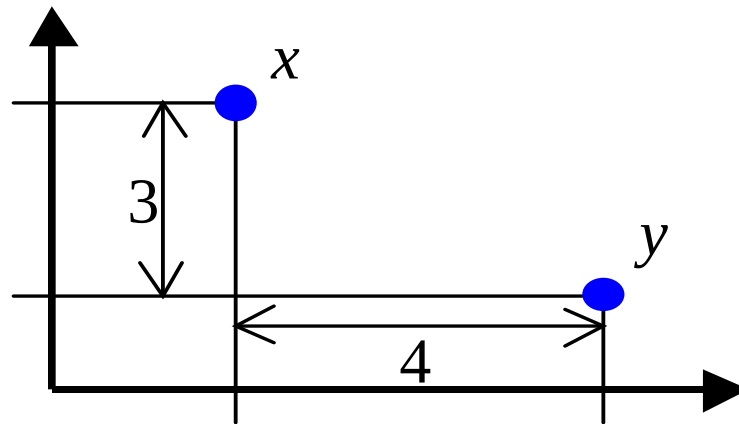
$r = 1$ (Manhattan distance)

$$d(x, y) = \sum_{i=1}^p |x_i - y_i|$$

$r = +\infty$ ("sup" distance)

$$d(x, y) = \max_{1 \leq i \leq p} |x_i - y_i|$$

AN EXAMPLE



- 1: Euclidean distance :
- 2: Manhattan distance :
- 3: "sup" distance :

Hamming distance

- Manhattan distance is called *Hamming distance* when all features are binary.
 - Minimum number of *substitutions* required to change one string into the other

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
<i>GeneA</i>	0	1	1	0	0	1	0	0	1	0	0	1	1	1	0	0	1
<i>GeneB</i>	0	1	1	1	0	0	0	0	1	1	1	1	1	1	0	1	1

Hamming Distance :

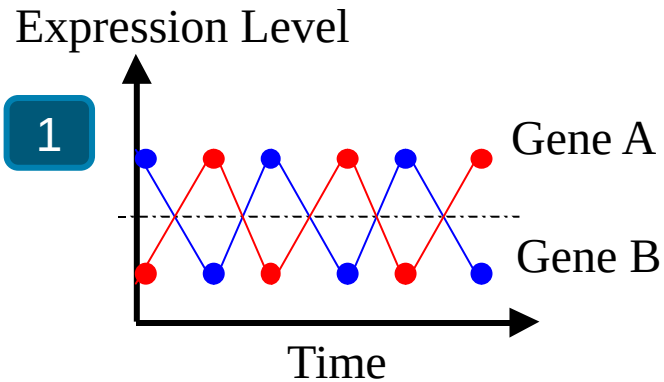
Edit Distance

- Protein sequences of {C,A,T,G}.
- Distance between sequences is *edit distance*, the minimum number of inserts and deletes needed to turn one into the other.
- Note there is a “distance,” but no convenient space in which points “live.”

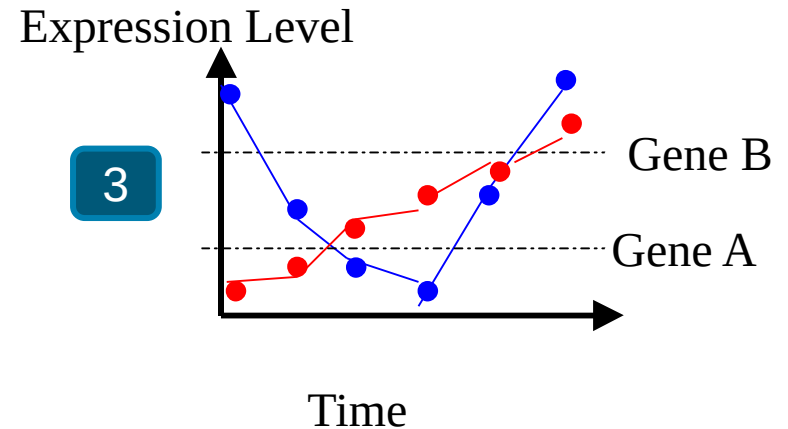
*	B	I	O	G	R	A	P	H	Y
A	U	T	O	G	R	A	P	H	*
i	s	s							d

SIMILARITY MEASURES: CORRELATION COEFFICIENT

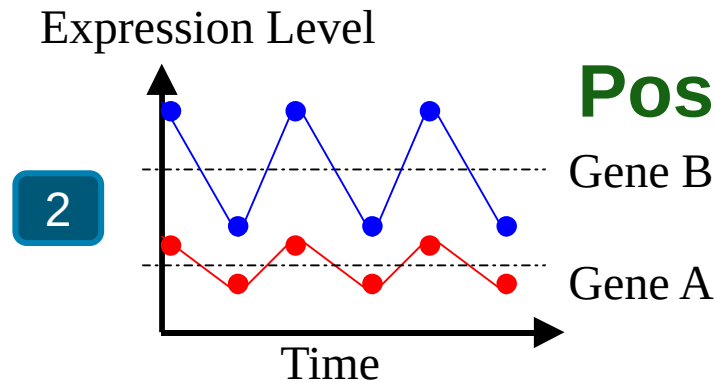
Negatively correlated



Uncorrelated



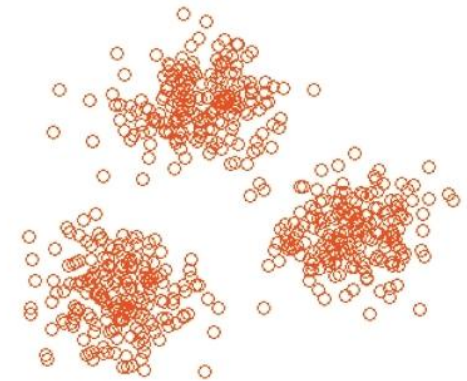
Positively correlated



Overview: Methods of Clustering

- Point assignment:

- Maintain a set of clusters
- Points belong to “nearest” cluster



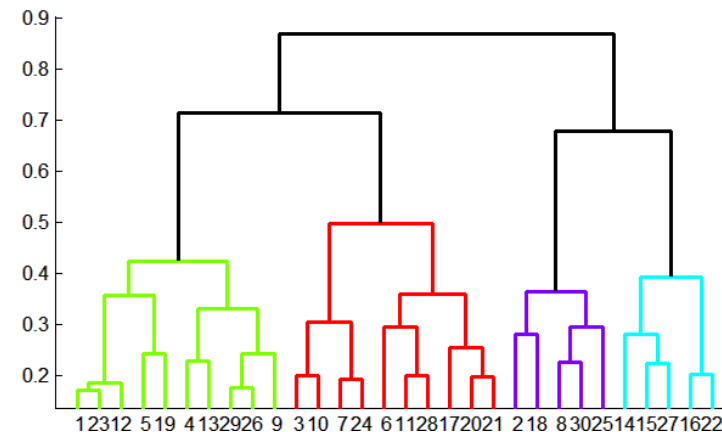
- Hierarchical:

- Agglomerative (bottom up):

- Initially, each point is a cluster
- Repeatedly combine the two “nearest” clusters into one

- Divisive (top down):

- Start with one cluster and recursively split it



k-means

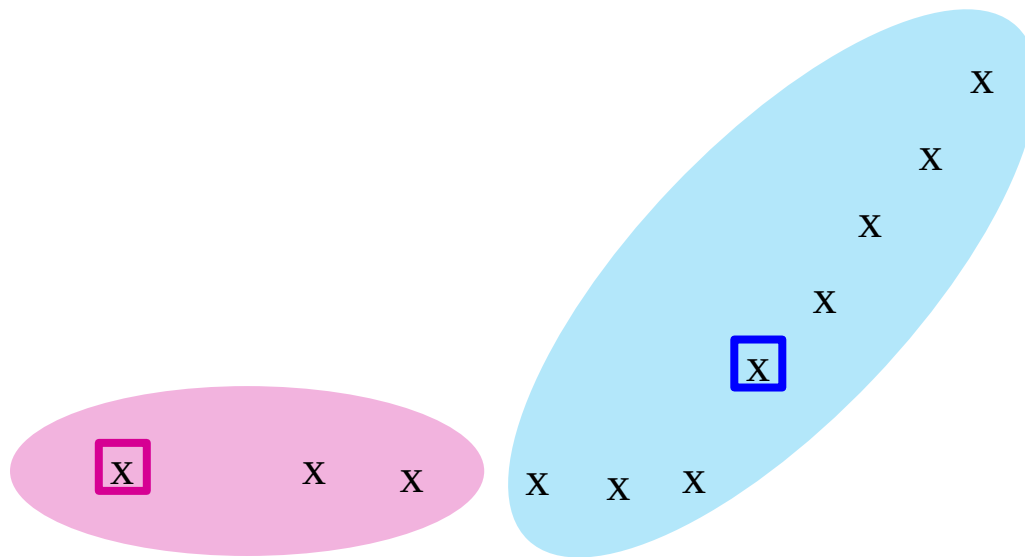
k -means Algorithm(s)

- Assumes Euclidean distance
- Start by picking k , the number of clusters
- Initialize clusters by picking one point per cluster
 - **Example:** Pick one point at random, then $k-1$ other points, each as far away as possible from the previous points

Populating Clusters

- **1)** For each point, place it in the cluster whose current centroid it is nearest
- **2)** After all points are assigned, update the locations of centroids of the k clusters
- **3)** Reassign all points to their closest centroid
 - Sometimes moves points between clusters
- **Repeat 2 and 3 until convergence**
 - **Convergence:** Points don't move between clusters and centroids stabilize

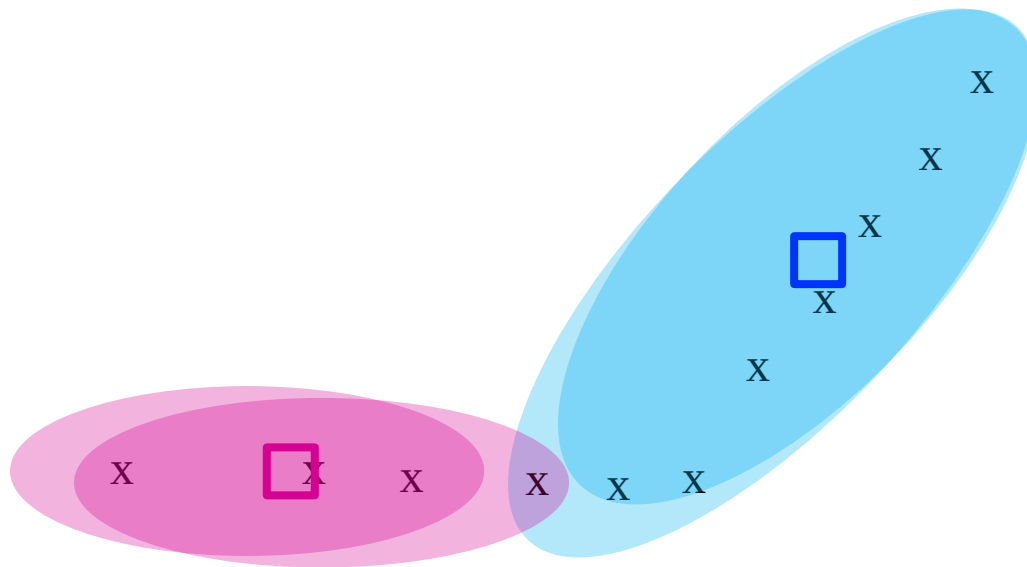
EXAMPLE: ASSIGNING CLUSTERS



x ... data point
□ ... centroid

Clusters after round 1

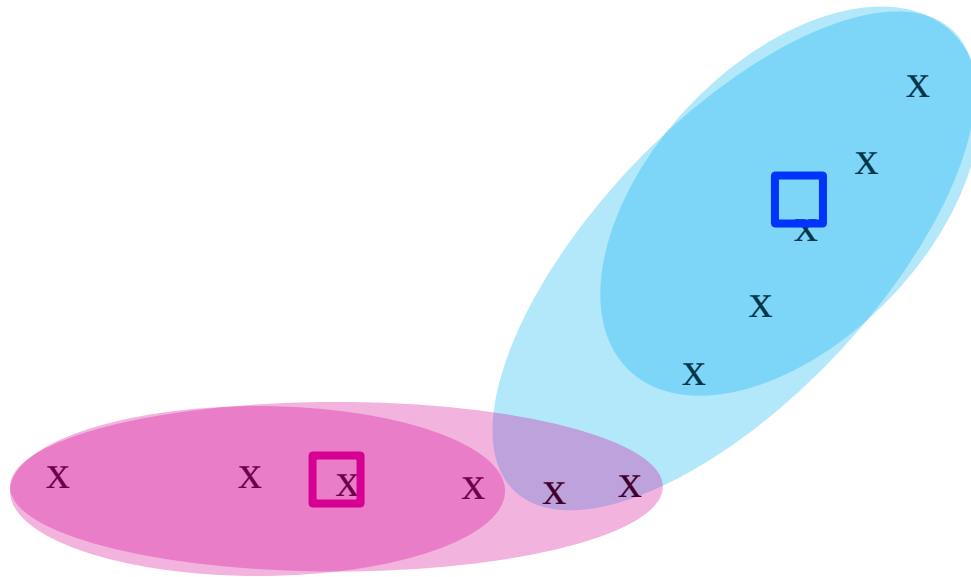
EXAMPLE: ASSIGNING CLUSTERS



x ... data point
□ ... centroid

Clusters after round 2

EXAMPLE: ASSIGNING CLUSTERS



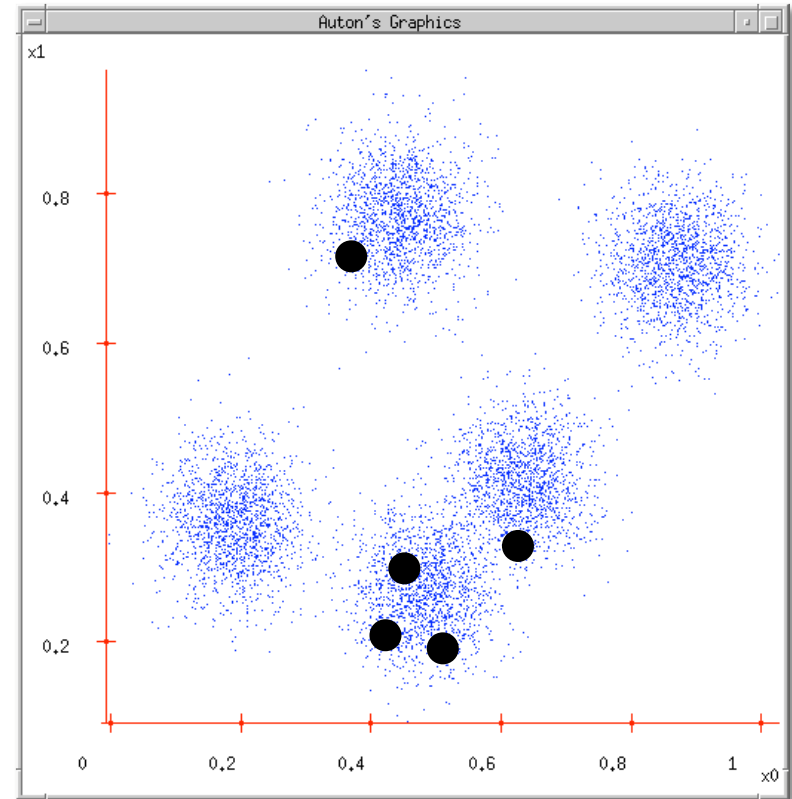
x ... data point

□ ... centroid

Clusters at the end

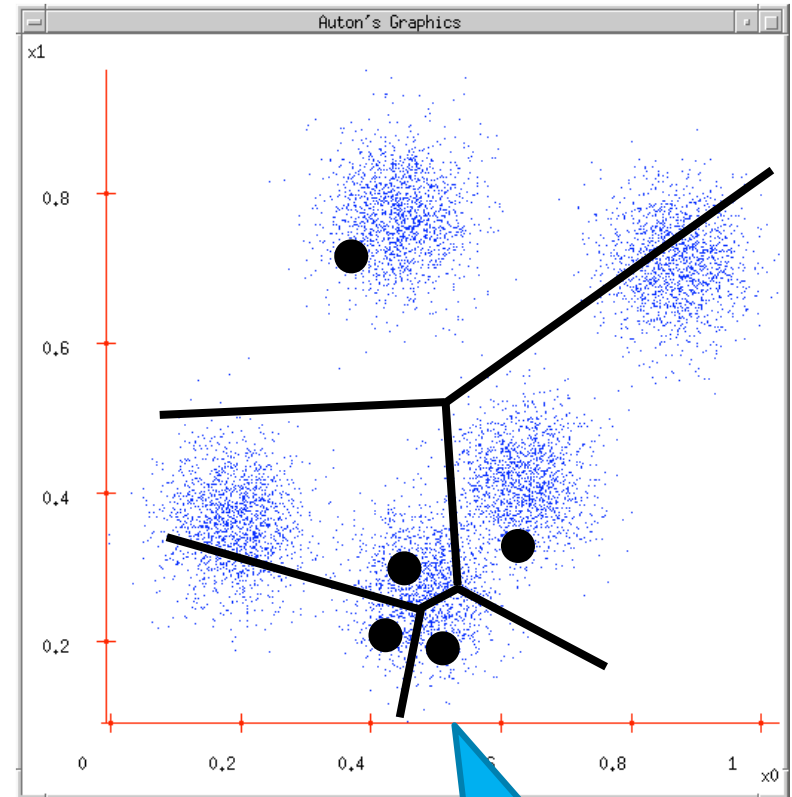
K-Means (k , data)

- Randomly choose k cluster center locations (centroids)
- Loop until convergence
 - Assign each point to the cluster of the closest centroid
 - Re-estimate the cluster centroids based on the data assigned to each
- Convergence: no point is assigned to a different cluster



K-Means (k , data)

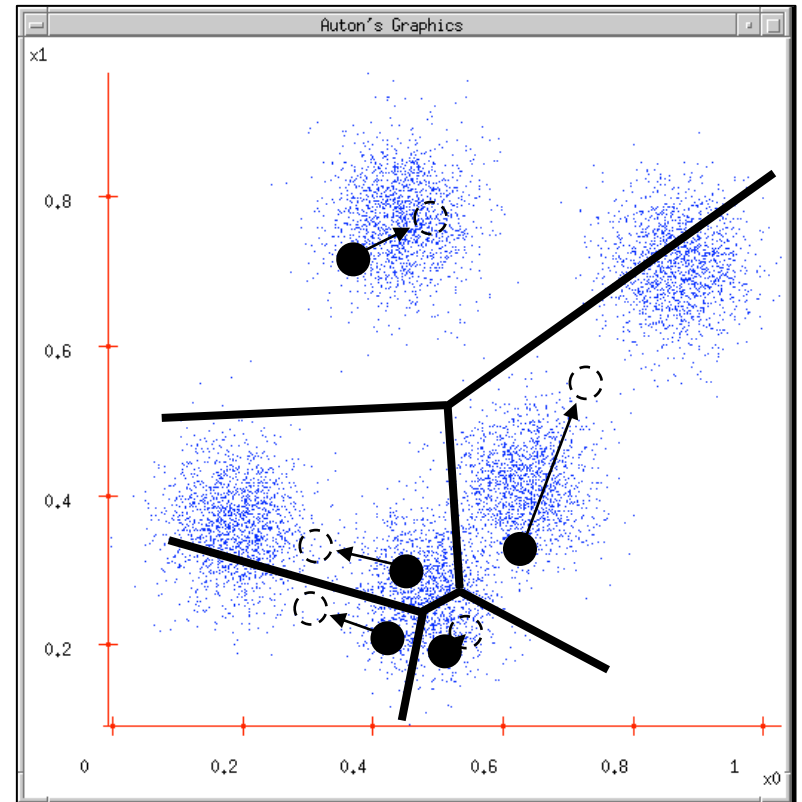
- Randomly choose k cluster center locations (centroids)
- Loop until convergence
 - Assign each point to the cluster of the closest centroid.
 - Re-estimate the cluster centroids based on the data assigned to each
- Convergence: no point is assigned to a different cluster



Voronoi
regions

K-Means (k , data)

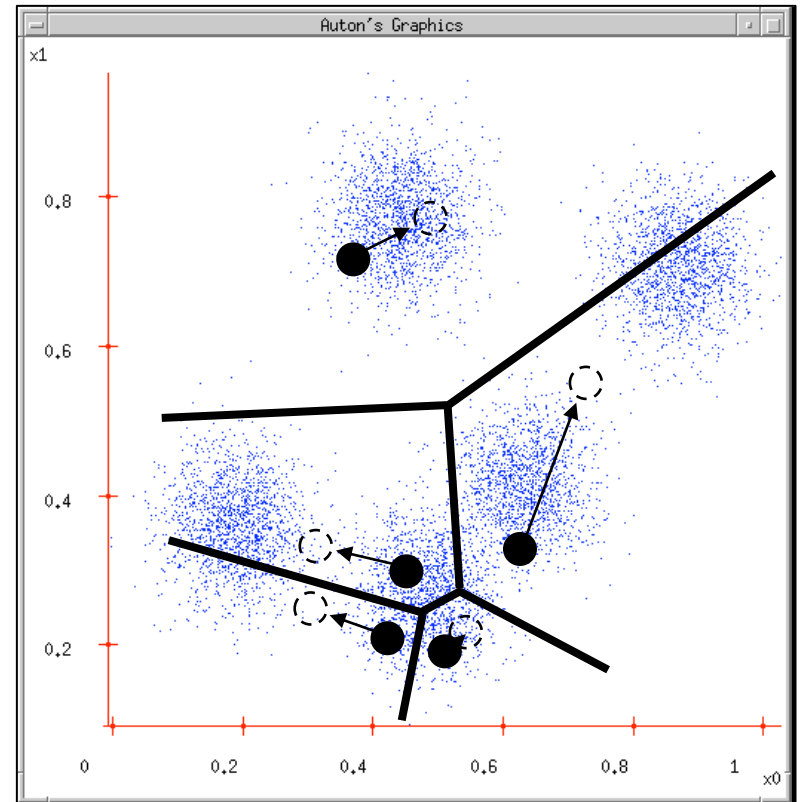
- Randomly choose k cluster center locations (centroids)
- Loop until convergence
 - Assign each point to the cluster of the closest centroid
 - Re-estimate the cluster centroids based on the data assigned to each
- Convergence: no point is assigned to a different cluster



K-Means (k , data)

- Randomly choose k cluster center locations (centroids)
- Loop until convergence
 - Assign each point to the cluster of the closest centroid
 - Re-estimate the cluster centroids based on the data assigned to each
- Convergence: no point is assigned to a different cluster

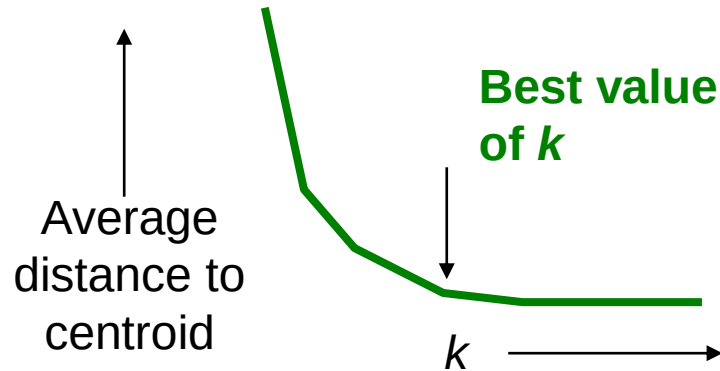
Why?



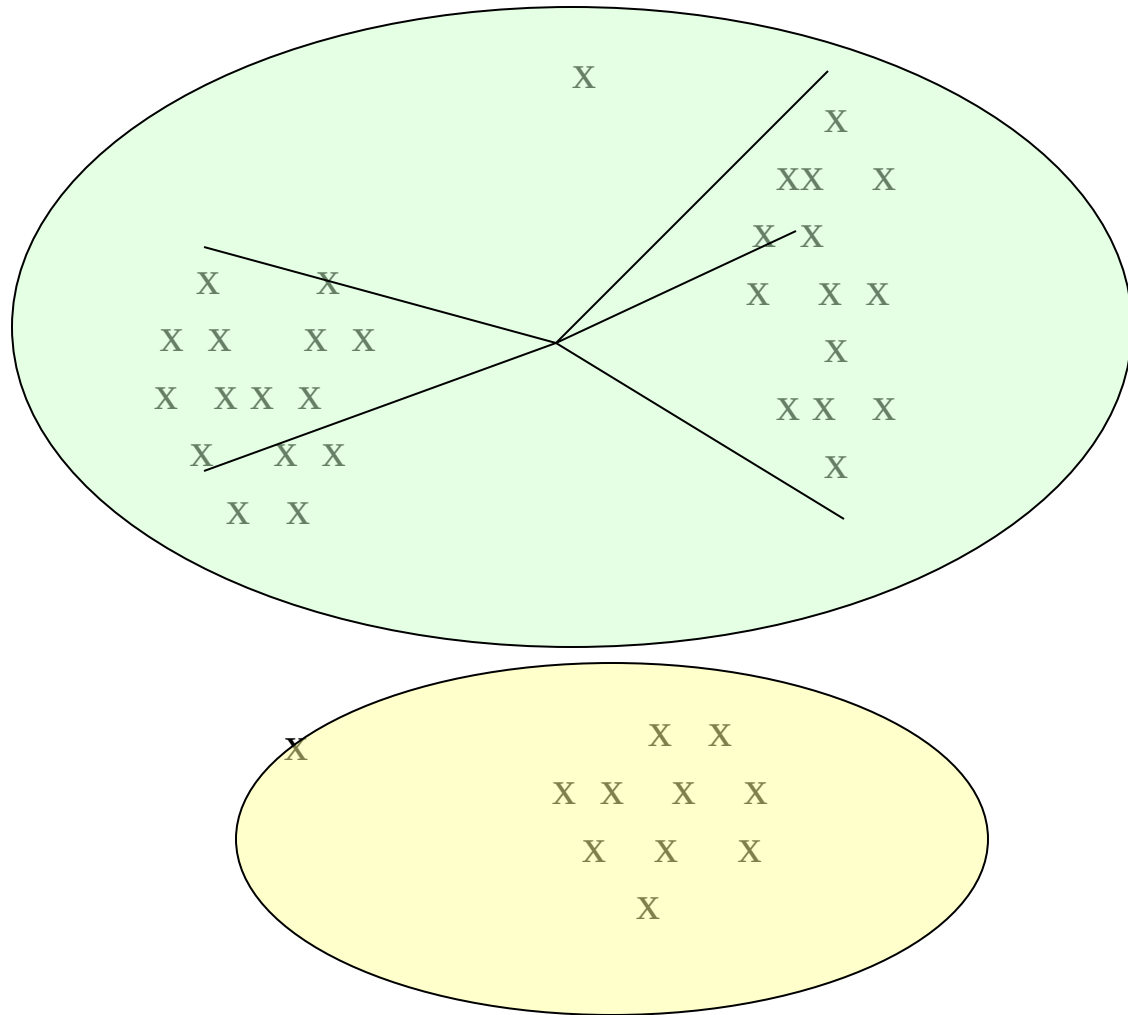
Getting the k right

How to select k ?

- Try different k , looking at the change in the average distance to centroid as k increases
- Average falls rapidly until right k , then changes little

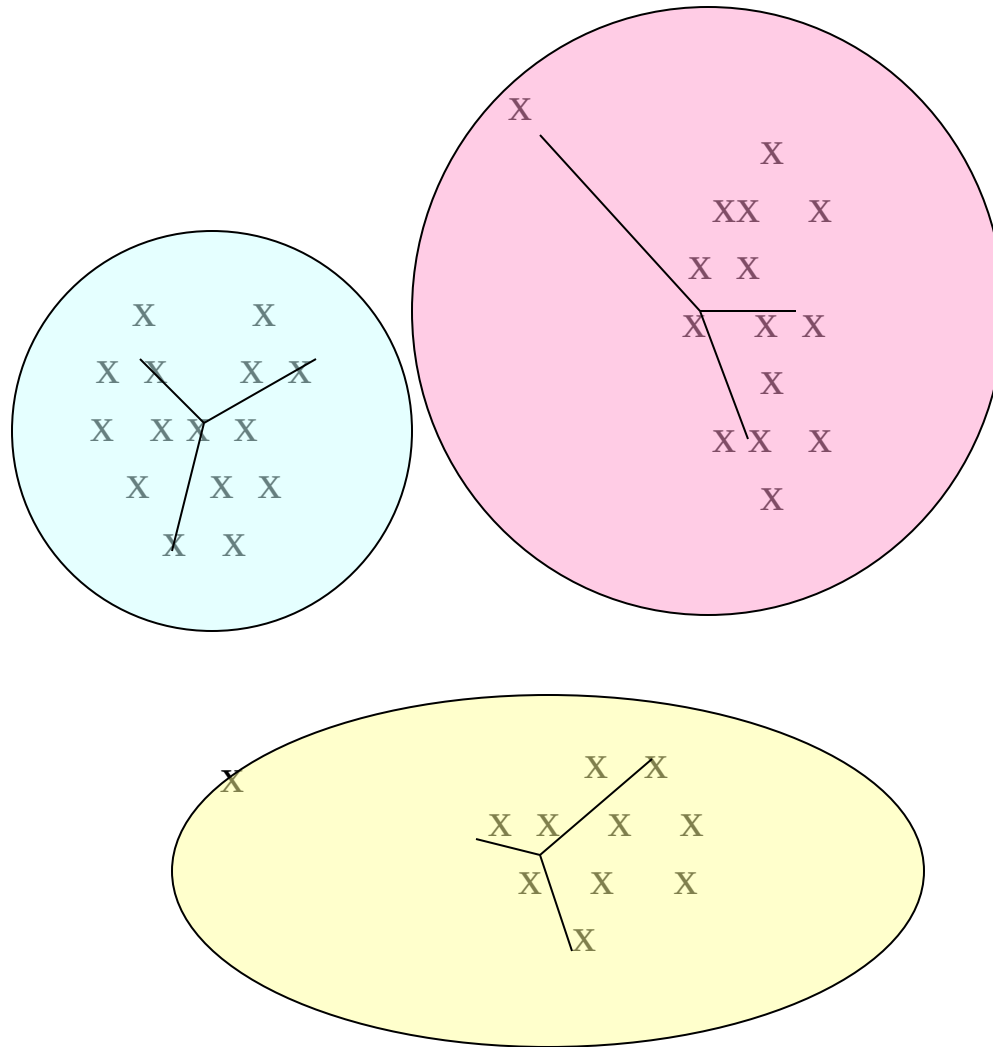


Too few;
many long
distances
to centroid.



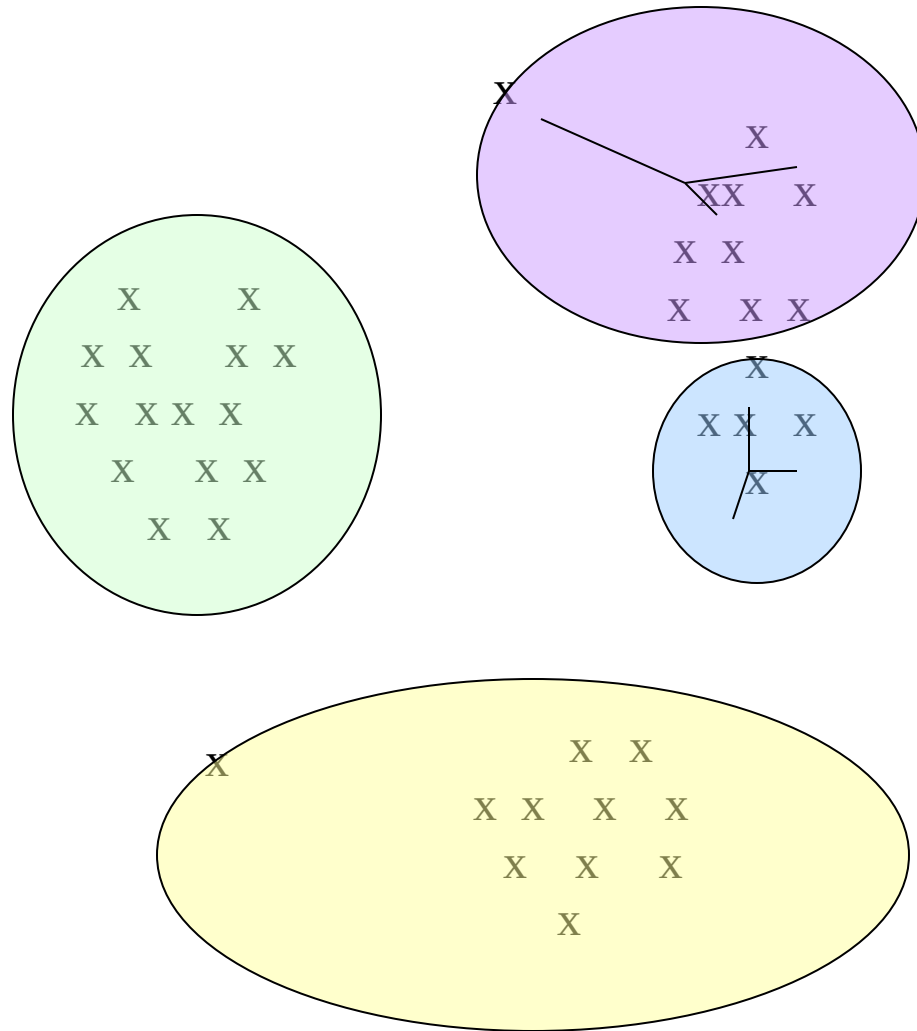
EXAMPLE: PICKING K

Just right;
distances
rather short.



EXAMPLE: PICKING K

Too many;
little improvement
in average
distance.



Problems with K-Means

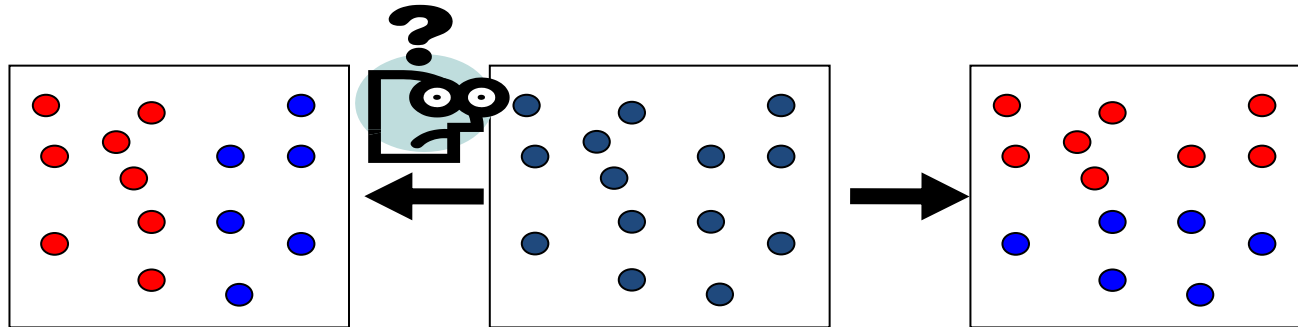
- Only works for **numeric** data (typical reals)
- **Very** sensitive to the **initial points**
 - Do many runs of k-Means with different initial centroids
 - Seed the centroids using a better method than random. (e.g., Farthest-first sampling)
 - Sensitive to outliers.
- Must **manually choose k**

Seed Choice: K-Means++

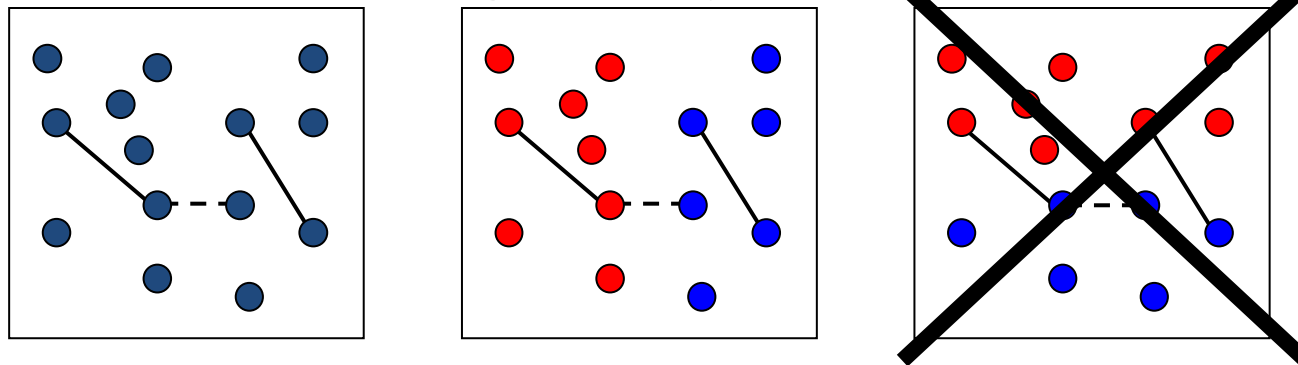
- K-Means results can vary based on random seed selection.
- **K-means++**: choose each point at random, with probability proportional to its squared distance from the centers chosen already.
 - Choose one center uniformly at random among given datapoints.
 - For each data point x , compute $D(x)$
 $D(x) = \text{distance}(x, \text{nearest center})$
 - Choose one new data point at random as a new center
 $P(x) \propto D(x)^2$.
 - Repeat Steps 2 and 3 until k centers have been chosen.
- Run standard K-Means with this centroid initialization.

Problems with K-Means

- How do you tell it which clustering you want?



– Constrained clustering techniques



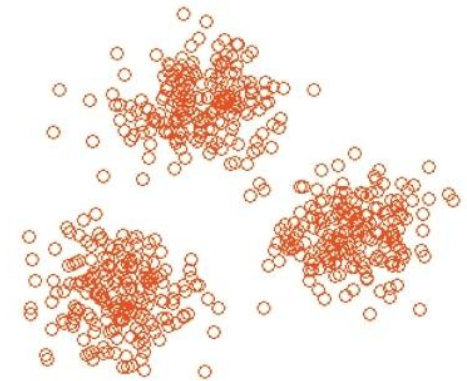
— Same-cluster constraint
(must-link)

- - - Different-cluster constraint
(cannot-link)

Overview: Methods of Clustering

- Point assignment:

- Maintain a set of clusters
- Points belong to “nearest” cluster



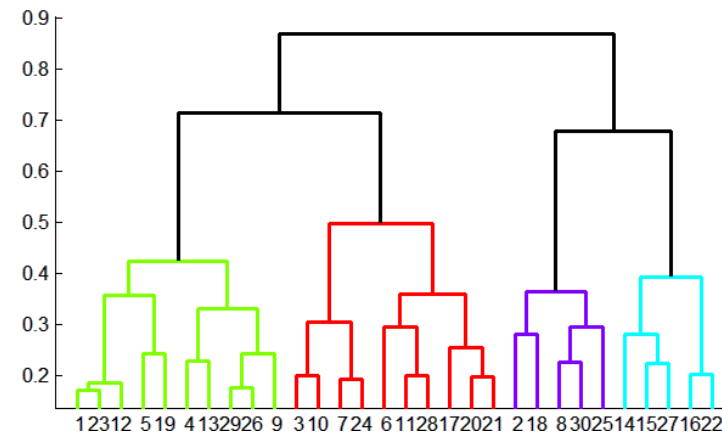
- Hierarchical:

- Agglomerative (bottom up):

- Initially, each point is a cluster
- Repeatedly combine the two “nearest” clusters into one

- Divisive (top down):

- Start with one cluster and recursively split it

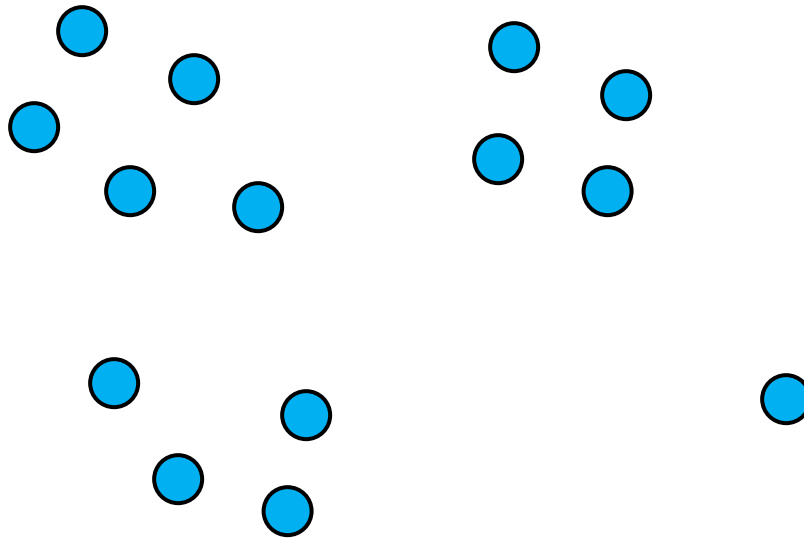


Hierarchical Clustering

Divisive hierarchical clustering

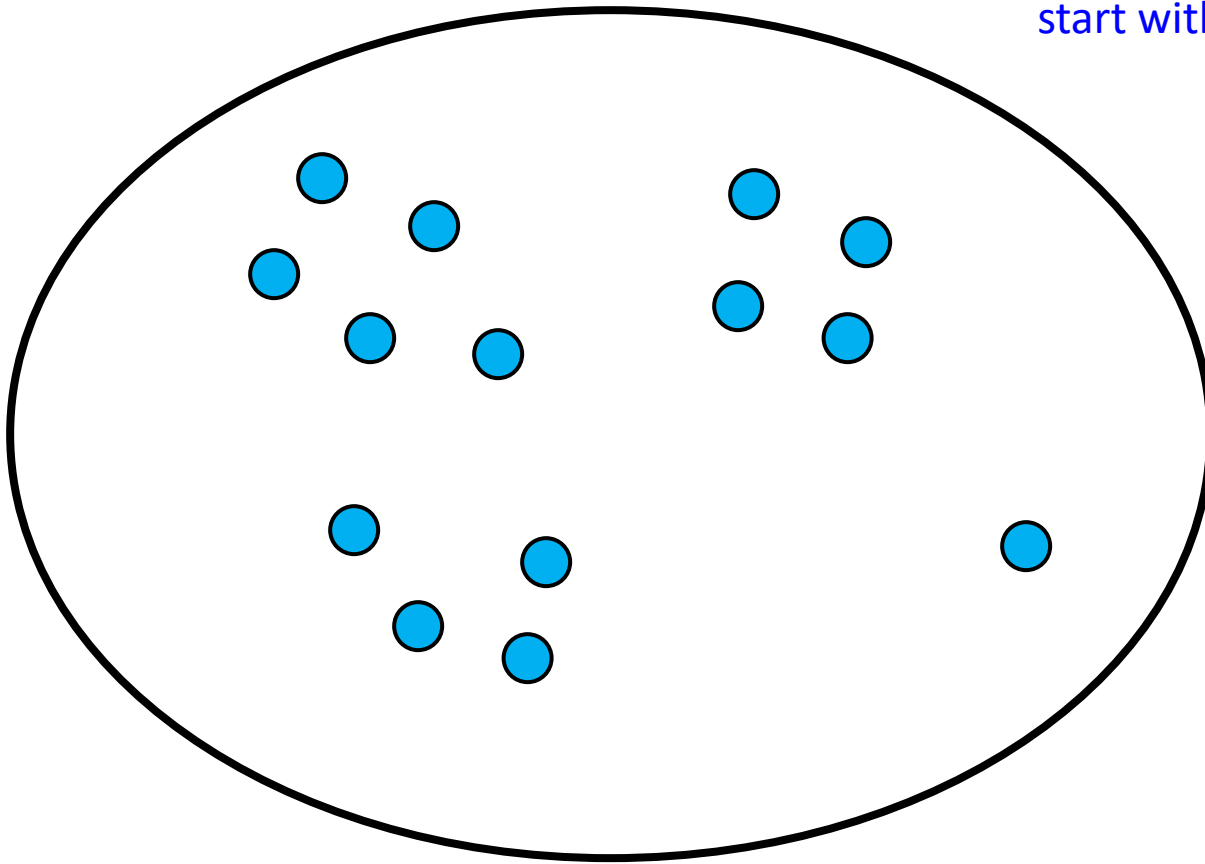
- Top-down
- Common approach:
 - Let $\mathbf{C}$ be a set of clusters
 - Initialize $\mathbf{C}$ to be the one-clustering of the data
 - While there exists a cluster c in $\mathbf{C}$
 - remove c from $\mathbf{C}$
 - partition c into 2 clusters using a flat clustering algorithm, c_1 and c_2
 - Add to c_1 and c_2 $\mathbf{C}$
- Bisecting k-means

Divisive clustering



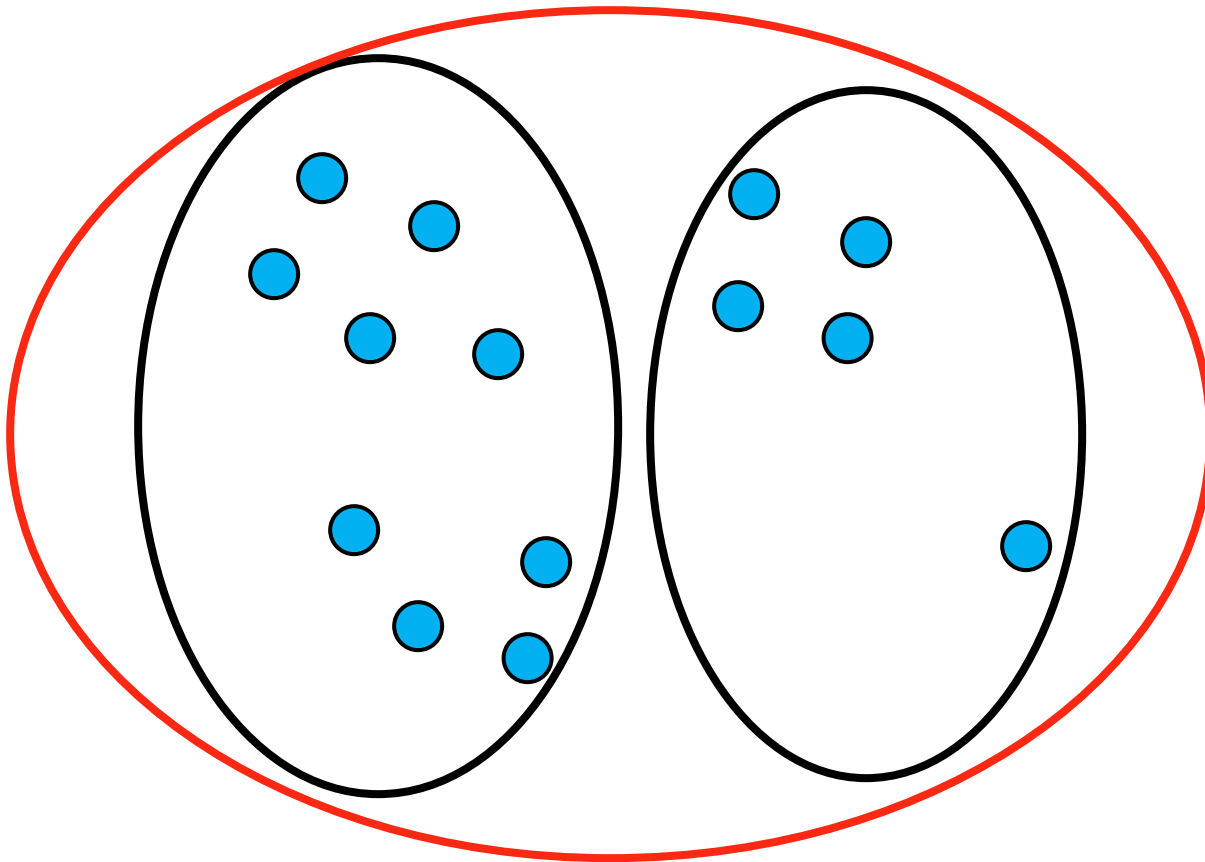
Divisive clustering

start with one cluster

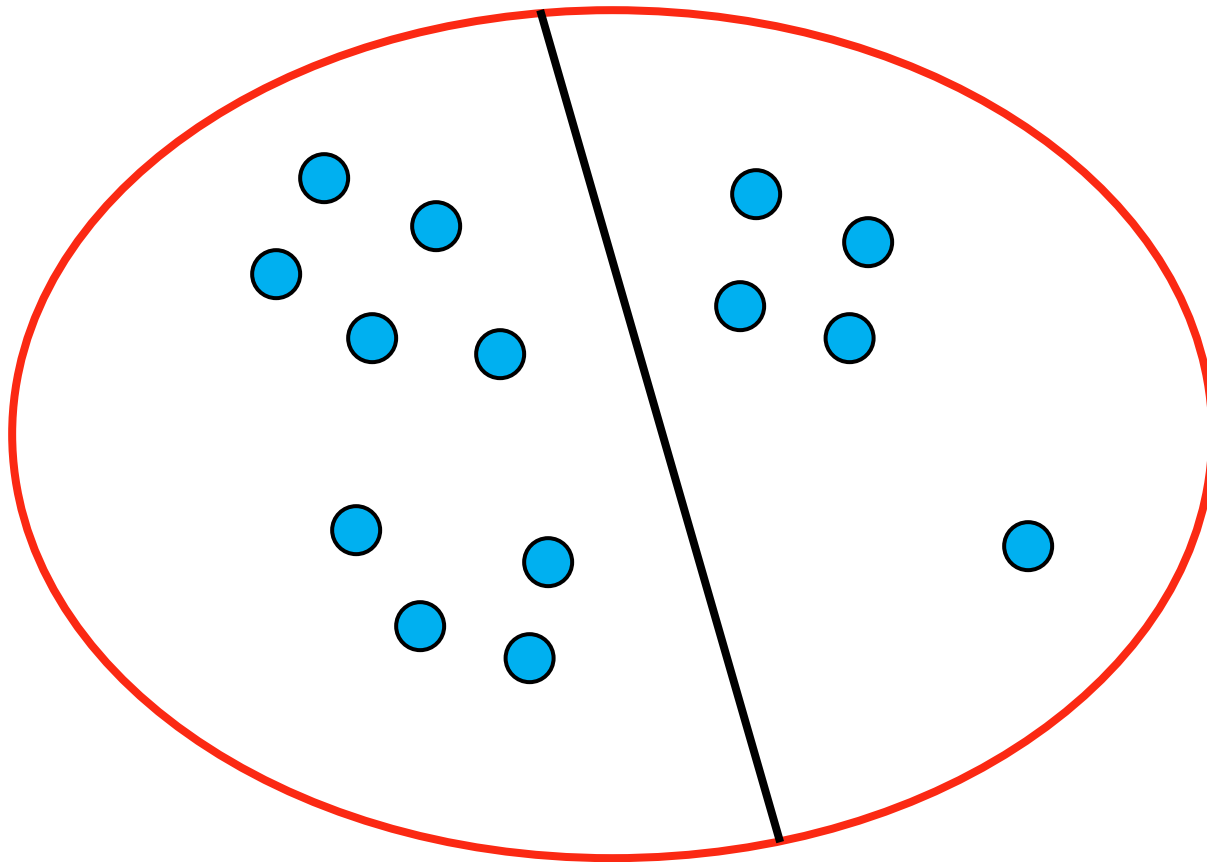


Divisive clustering

split using flat clustering, e.g. Kmeans

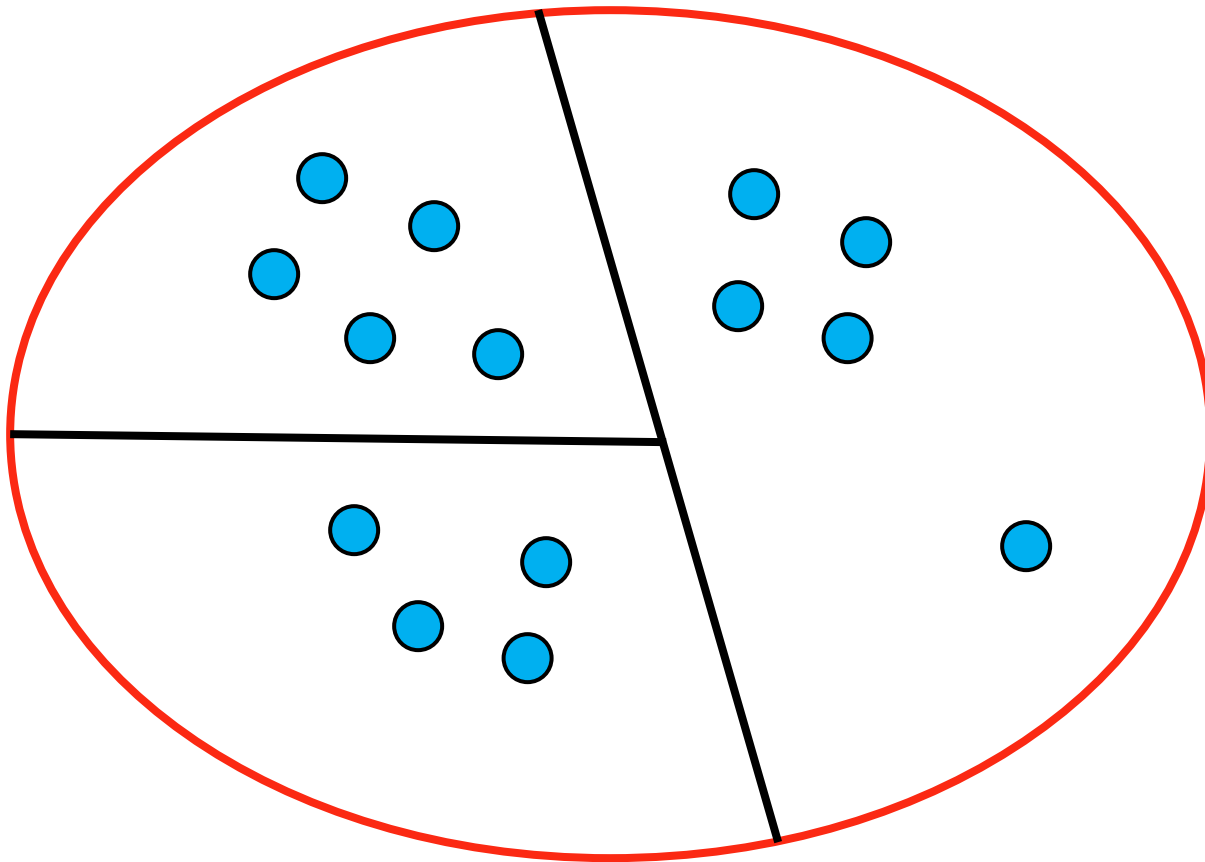


Divisive clustering



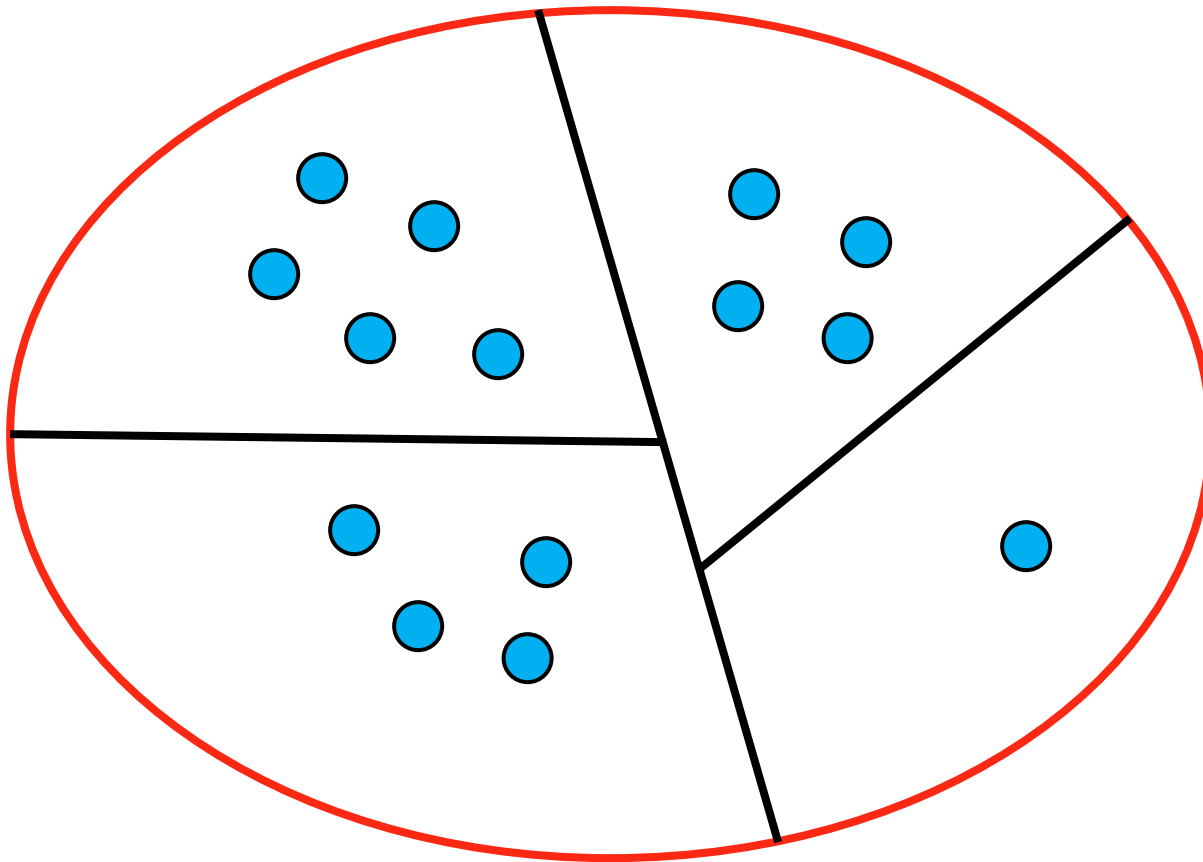
Divisive clustering

split using flat clustering

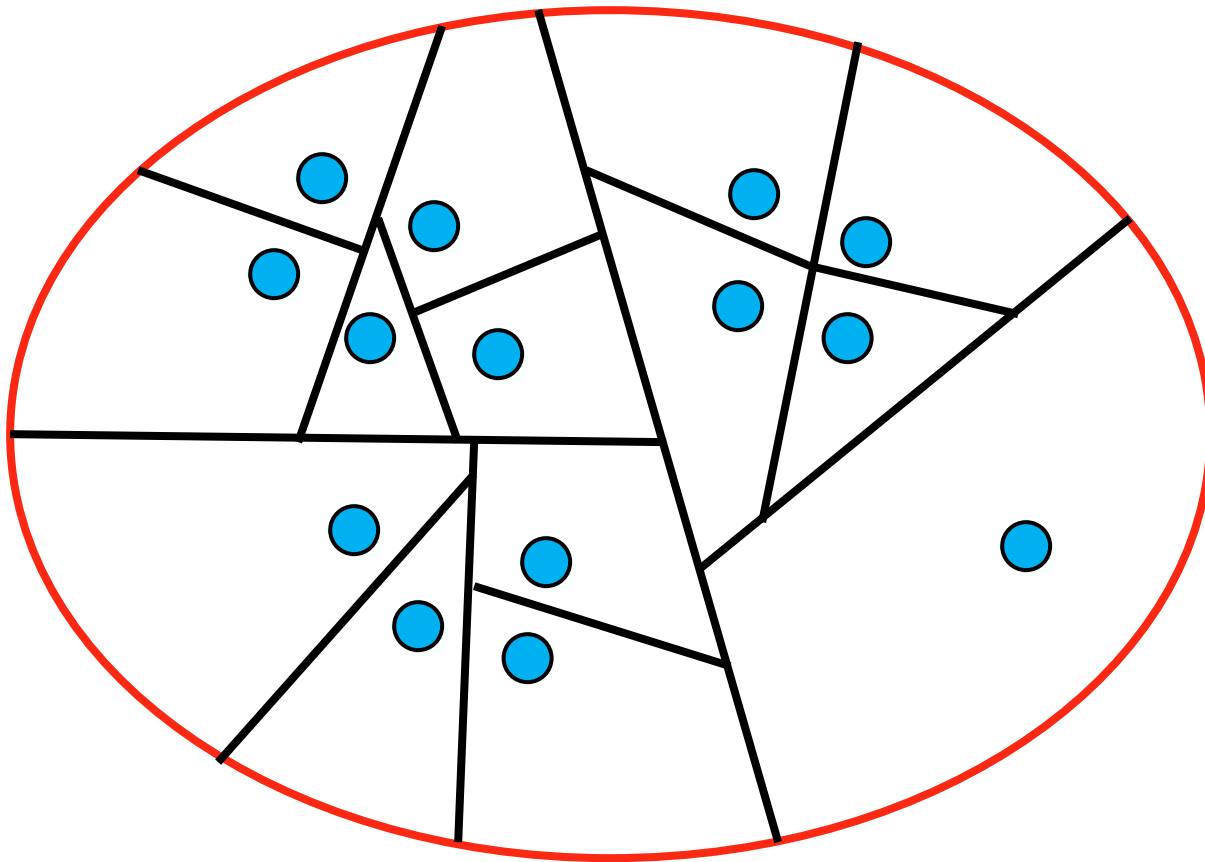


Divisive clustering

split using flat clustering

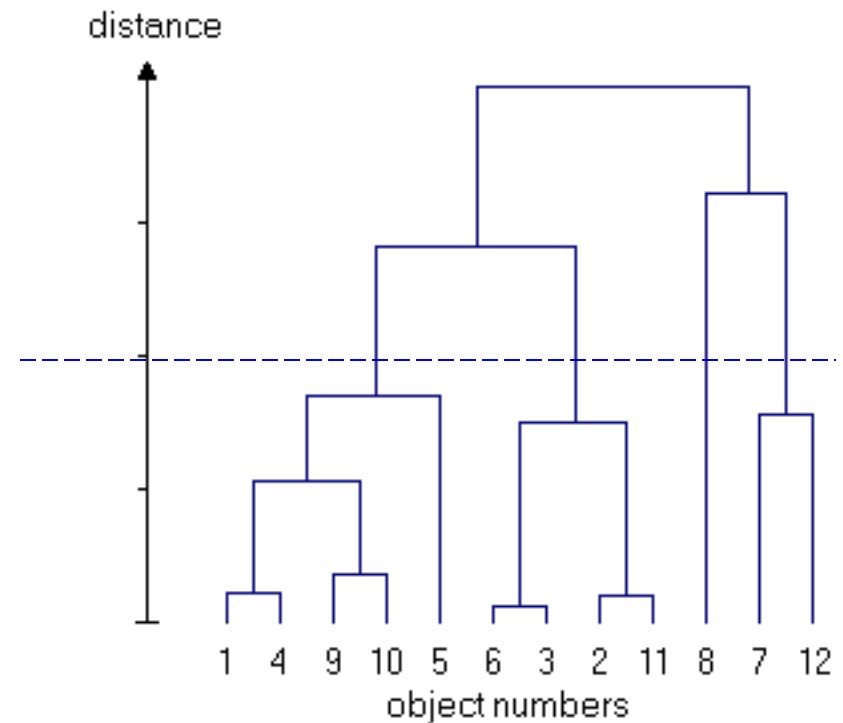


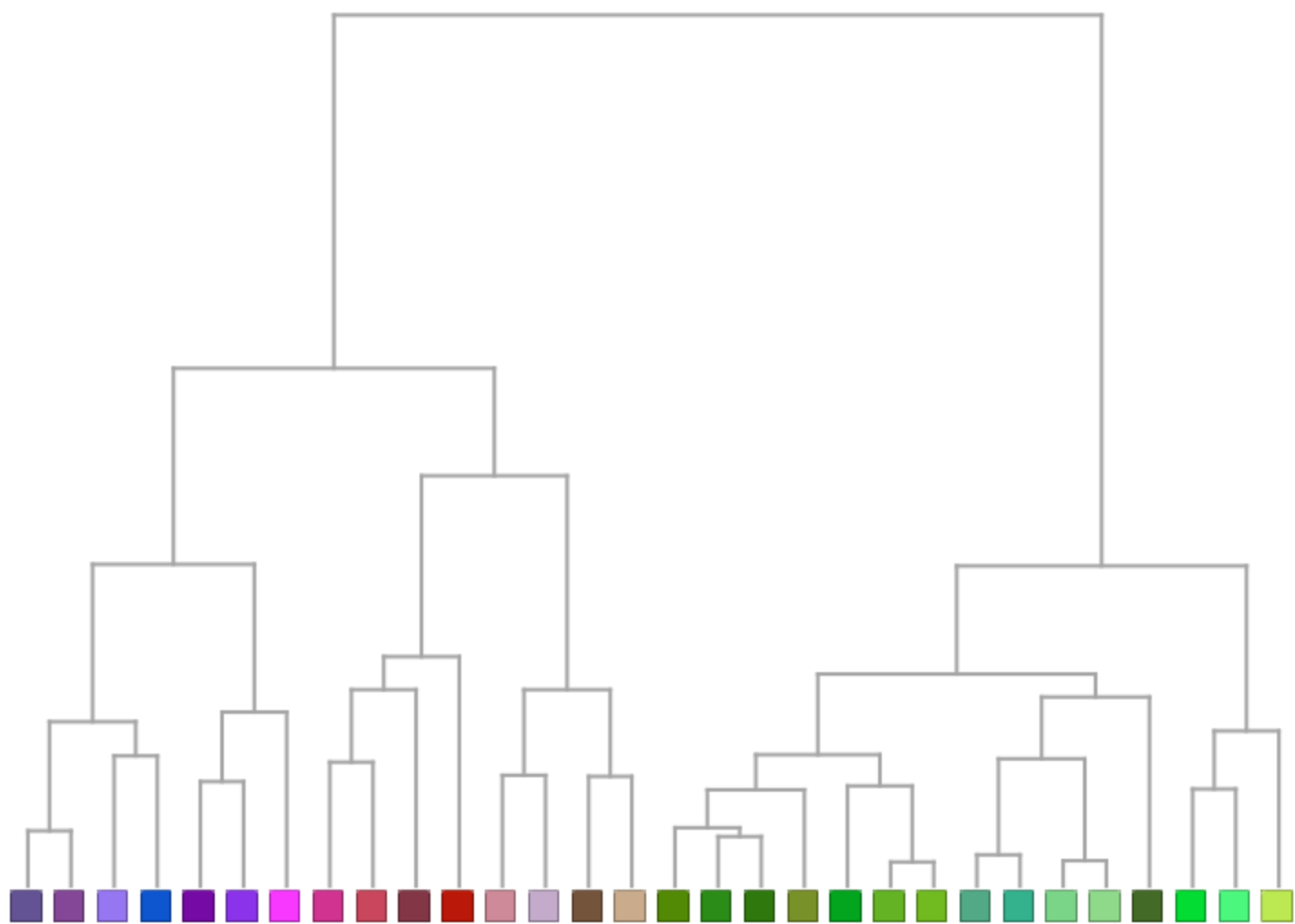
Divisive clustering



Hierarchical Clustering: Dendrogram

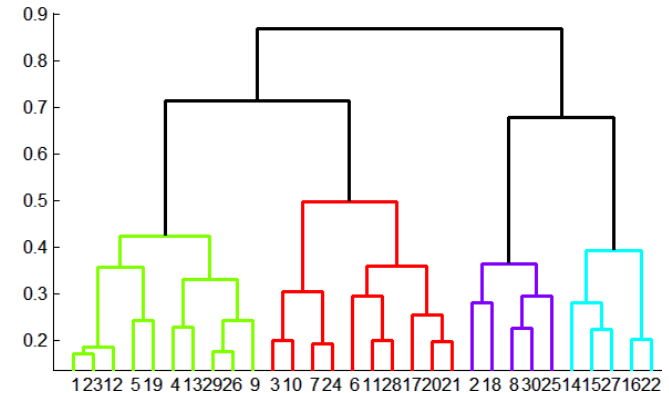
- Represents all partitionings of the data
- We can get a K clustering by looking at the **connected** components at any given level
- Frequently binary dendograms, but n-ary dendograms are easy to obtain with minor changes to algorithms





Hierarchical Clustering

- **Key operation:**
Repeatedly combine
two nearest clusters

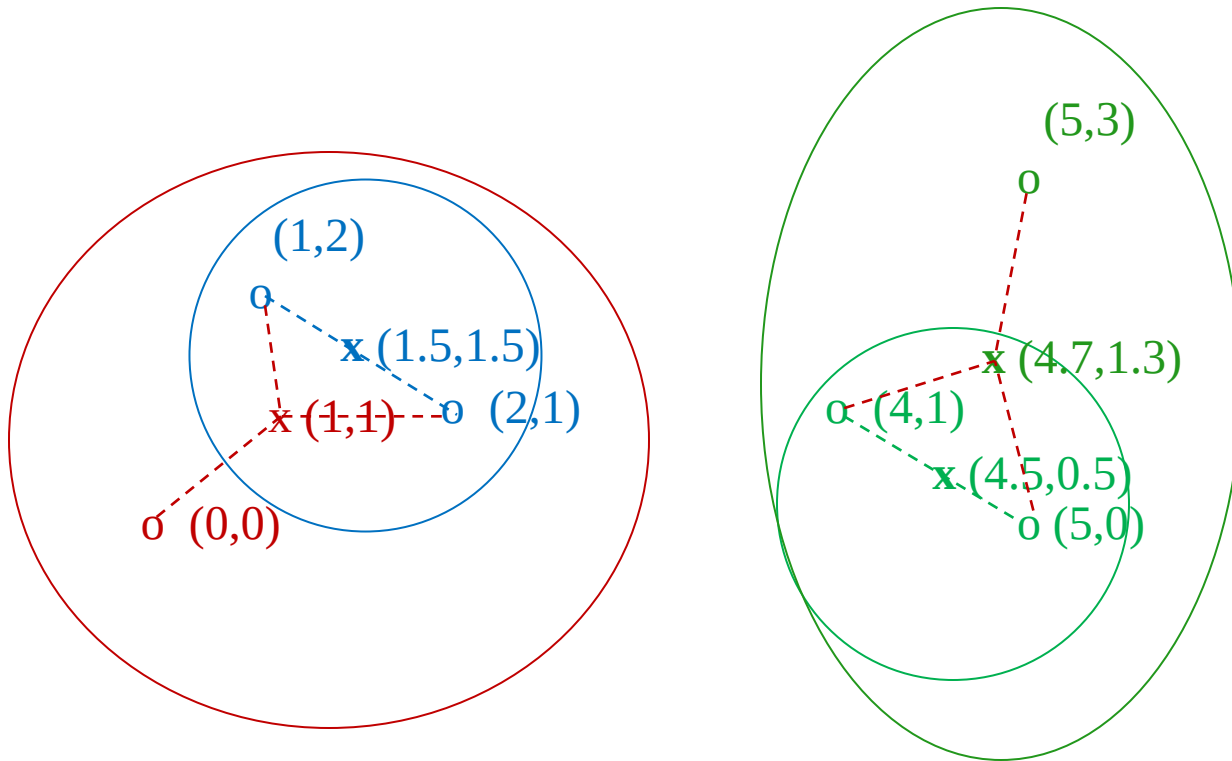


- **Three important questions:**
 - 1) How do you represent a cluster of more than one point?
 - 2) How do you determine the “nearness” of clusters?
 - 3) When to stop combining clusters?

Hierarchical Clustering

- **Key operation:** Repeatedly combine two nearest clusters
- **(1) How to represent a cluster of many points?**
 - **Key problem:** As you merge clusters, how do you represent the “location” of each cluster, to tell which pair of clusters is closest?
- **Euclidean case:** each cluster has a **centroid** = average of its (data)points
- **(2) How to determine “nearness” of clusters?**
 - Measure cluster distances by distances of centroids

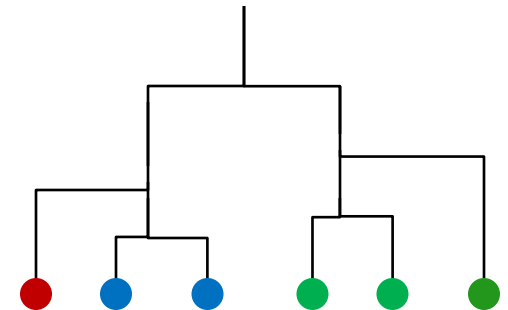
EXAMPLE: HIERARCHICAL CLUSTERING



Data:

σ ... data point

$\bar{x}$... centroid



Dendrogram

And in the Non-Euclidean Case?

- The only “locations” we can talk about are the points themselves
 - i.e., there is no “average” of two points
- **Approach 1:**
 - (1) How to represent a cluster of many points?
clustroid = (data)point “closest” to other points
 - (2) How do you determine the “nearness” of clusters? Treat clustroid as if it were centroid, when computing inter-cluster distances

“Closest” Point?

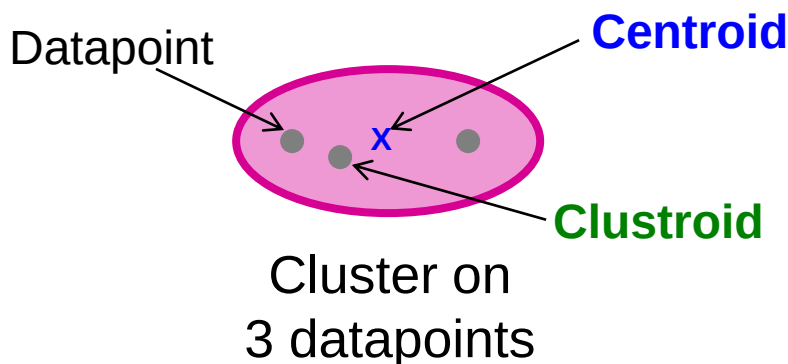
- (1) How to represent a cluster of many points?

clustroid = point “closest” to other points

- Possible meanings of “closest”:

- Smallest maximum distance to other points
- Smallest average distance to other points
- Smallest sum of squares of distances to other points

- For distance d clustroid c of cluster C is: $\min_c \sum_{x \in C} d(x, c)^2$



Centroid is the avg. of all (data)points in the cluster. This means centroid is an “artificial” point.

Clustroid is an **existing** (data)point that is “closest” to all other points in the cluster.

Defining “Nearness” of Clusters

- (2) How do you determine the “nearness” of clusters?

–Approach 2:

Intercluster distance = minimum of the distances between any two points, one from each cluster

–Approach 3:

Pick a notion of “**cohesion**” of clusters, *e.g.*, maximum distance from the clustroid

- Merge clusters whose *union* is most cohesive

Cohesion

- **Approach 3.1:** Use the **diameter** of the merged cluster = maximum distance between points in the cluster
- **Approach 3.2:** Use the **average distance** between points in the cluster
- **Approach 3.3:** Use a **density-based approach**
 - Take the diameter or avg. distance, e.g., and divide by the number of points in the cluster

Advantages of hierarchical clustering

- Don't need to specify the number of clusters
- Good for data visualization
 - See how the data points interact at many levels
 - Can view the data at multiple levels of granularity
 - Understand how all points interact
- Specifies all of the K clusterings/partitions

Implementation

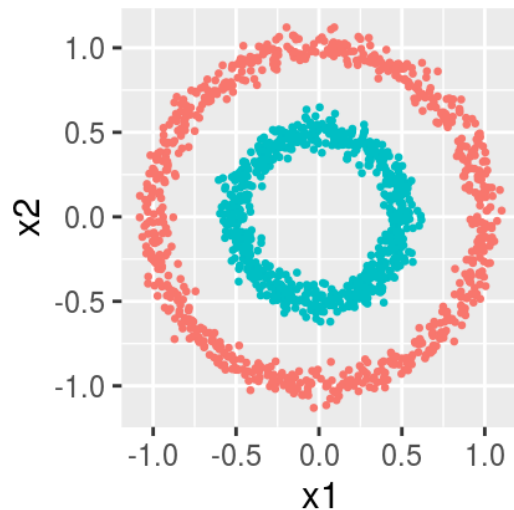
- **Naïve implementation of hierarchical clustering:**
 - At each step, compute pairwise distances between all pairs of clusters, then merge
 - $O(N^3)$
- Careful implementation using priority queue can reduce time to $O(N^2 \log N)$
 - **Still too expensive for really big datasets that do not fit in memory**

Density-Based Clustering

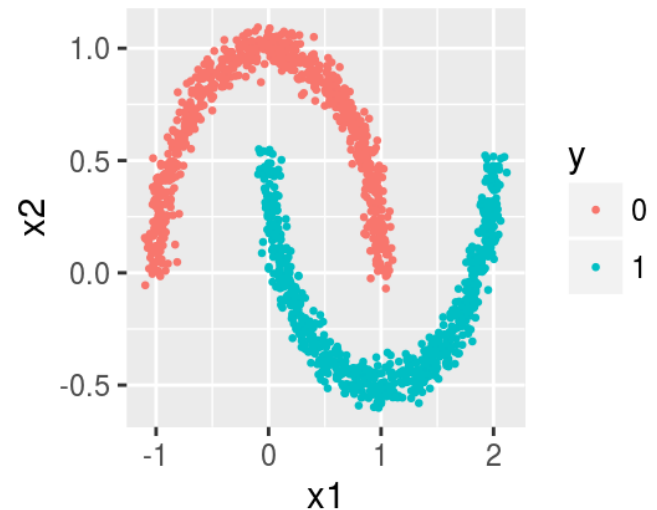
What would the clustering algos we discussed do here?



Circles



Moons



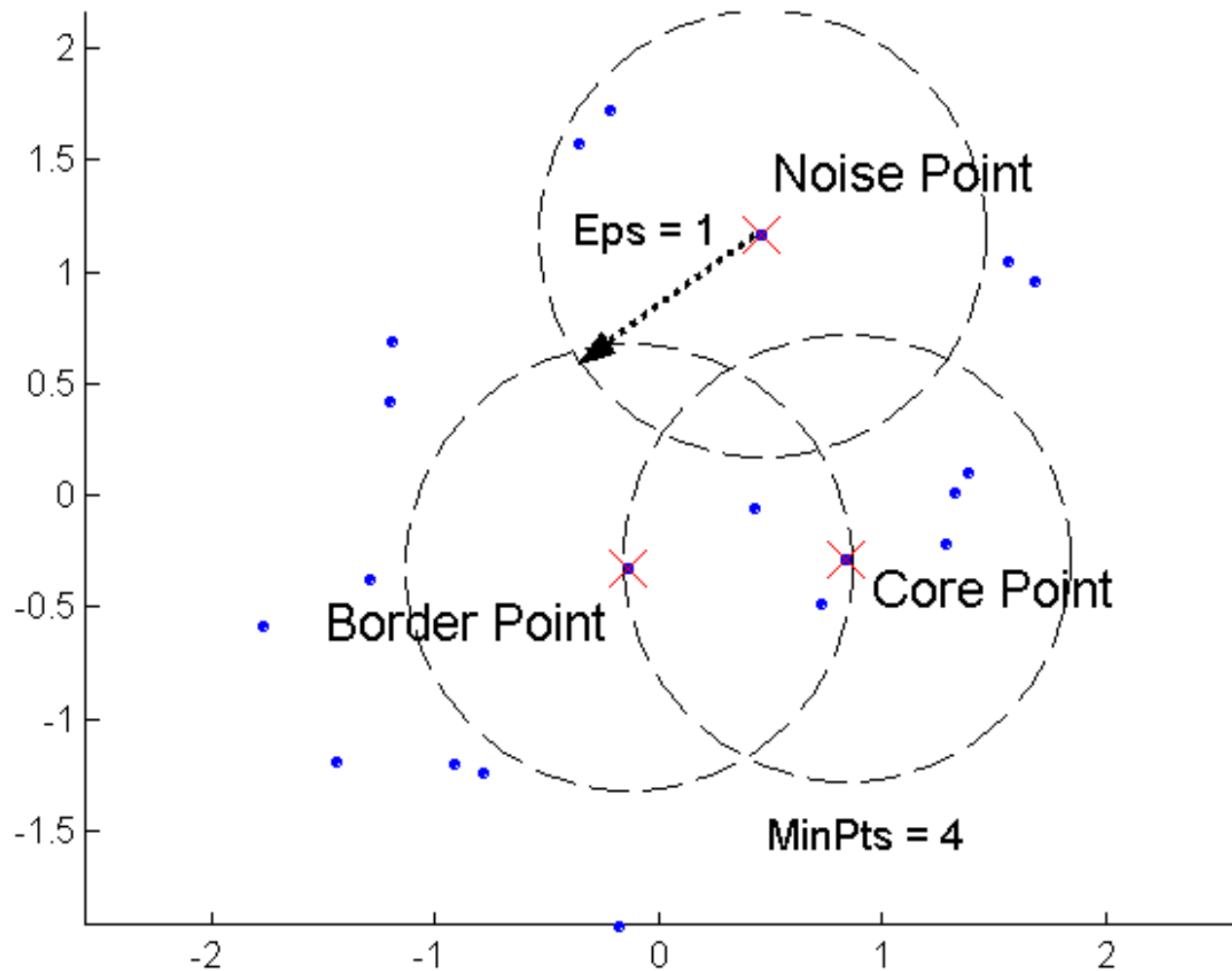
Density-Based Clustering Methods

- Clustering based on **density** (local cluster criterion), such as density-connected points or based on an explicitly constructed density function
- Major features:
 - Discover clusters of **arbitrary shape**
 - Handle **noise**

DBSCAN

- Two parameters:
 - ***Eps***: Maximum radius of the neighbourhood
 - ***MinPts***: Minimum number of points in an Eps-neighbourhood of that point

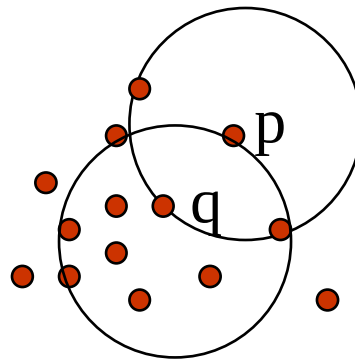
DBSCAN: Core, Border, and Noise Points



DBSCAN

- **Directly density-reachable**: A point p is directly density-reachable from a point q wrt. ***Eps***, ***MinPts*** if
 - 1) p belongs to $N_{Eps}(q)$
 - 2) core point condition:

$$|N_{Eps}(q)| \geq MinPts$$



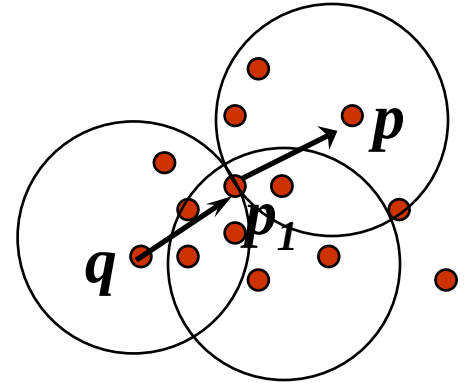
MinPts = 5

Eps = 1 cm

Density-Based Clustering

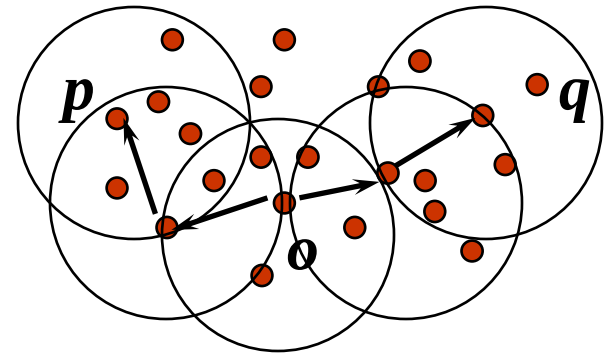
- Density-reachable:

- A point p is density-reachable from a point q wrt. Eps , $MinPts$ if there is a chain of points $p_1, \dots, p_n$, $p_1 = q$, $p_n = p$ such that p_{i+1} is directly density-reachable from p_i



- Density-connected:

- A point p is density-connected to a point q wrt. Eps , $MinPts$ if there is a point o such that both, p and q are density-reachable from o wrt. Eps and $MinPts$.



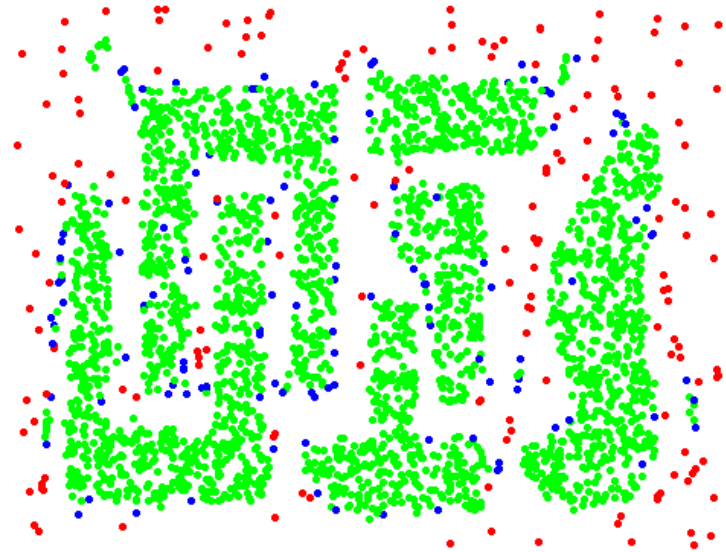
DBSCAN: The Algorithm

- Remove noise points
- Arbitrary select a core point p
- Retrieve all points density-reachable from p wrt ***Eps*** and ***MinPts***.
- A cluster is formed.
- Continue the process until all of the points have been processed.
- Check out https://youtu.be/_A9Tq6mGtLI?si=N6r16AOgTUvoT4pW&t=170

DBSCAN: Core, Border and Noise Points



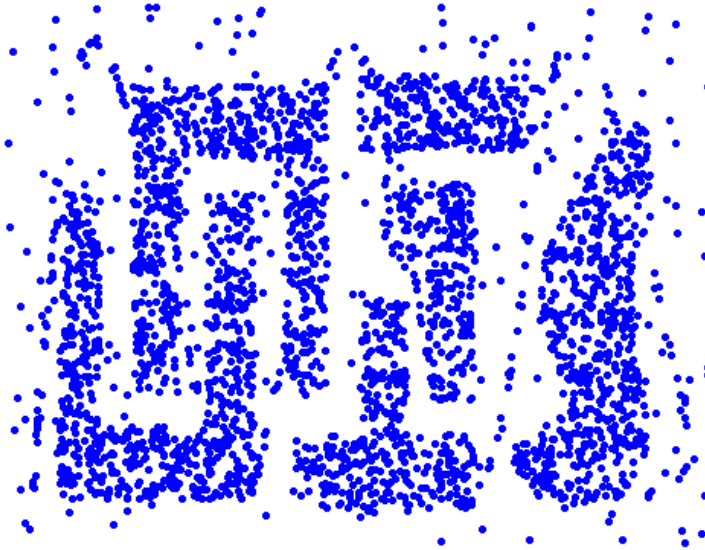
Original Points



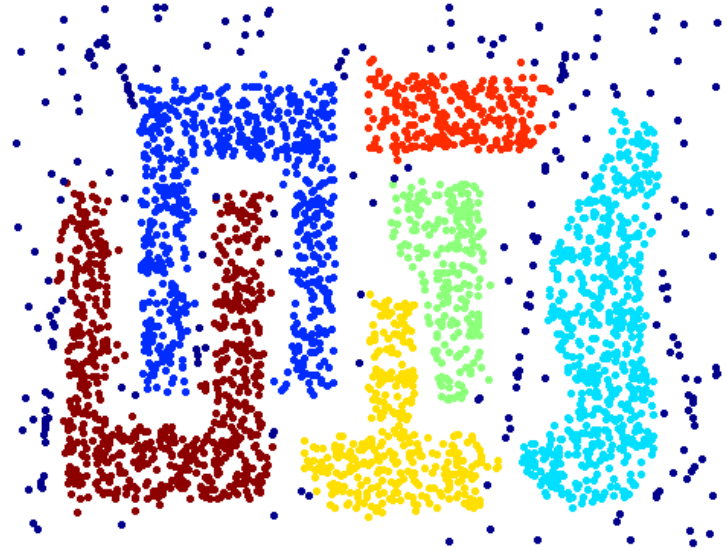
Point types: core,
border and noise

Eps = 10, MinPts = 4

When DBSCAN Works Well



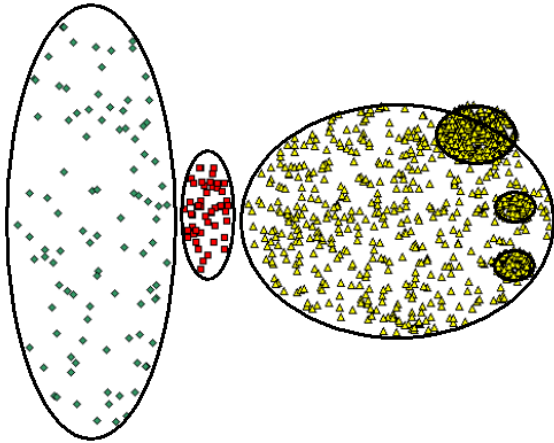
Original Points



Clusters

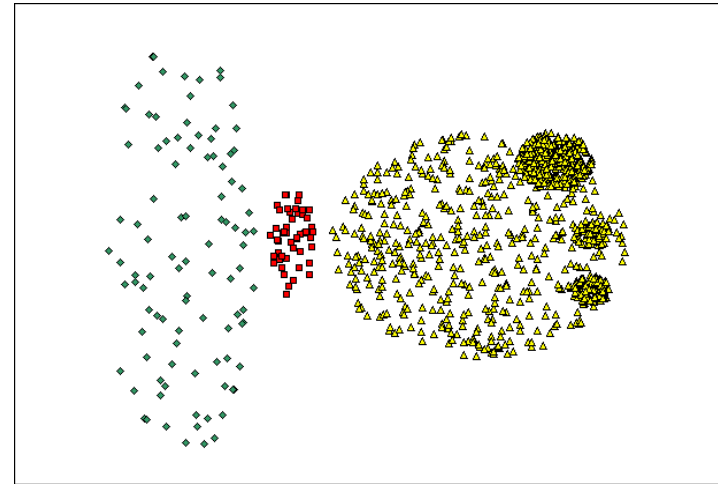
- Resistant to Noise
- Can handle clusters of different shapes and sizes

When DBSCAN Does NOT Work Well

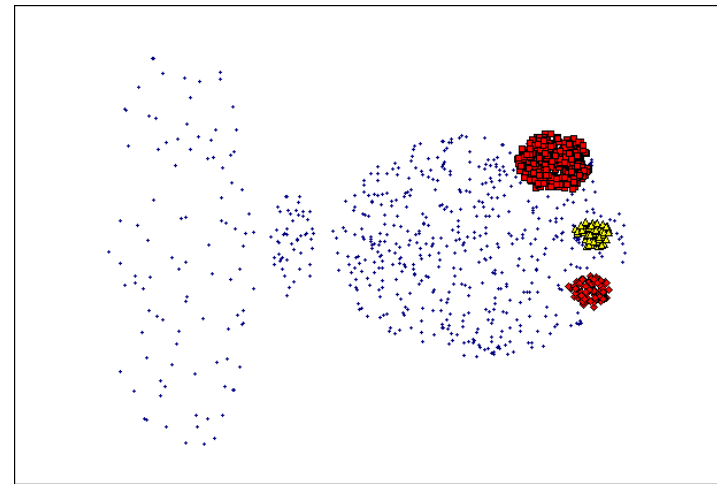


Original Points

- Varying densities
- High-dimensional data



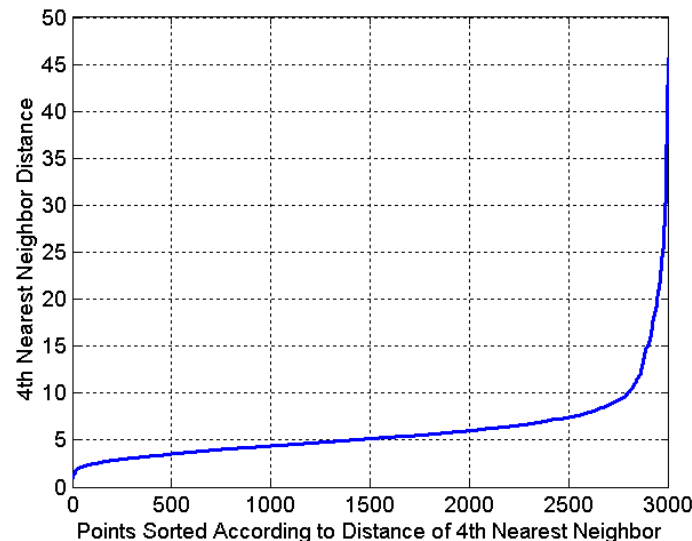
(MinPts=4, Eps=9.75).



(MinPts=4, Eps=9.92)

DBSCAN: Determining EPS and MinPts

- Idea is that for points in a cluster, their k^{th} nearest neighbors are at roughly the same distance
- Noise points have the k^{th} nearest neighbor at farther distance
- So, plot sorted distance of every point to its k^{th} nearest neighbor



~Evaluation

How good is your clustering?

- For supervised classification we have a variety of measures to evaluate how good our model is
 - Accuracy, precision, recall
- For cluster analysis, the analogous question is how to evaluate the “goodness” of the resulting clusters?
- But “clusters are in the eye of the beholder”!

“The validation of clustering structures is the most difficult and frustrating part of cluster analysis.

Without a strong effort in this direction, cluster analysis will remain a black art accessible only to those true believers who have experience and great courage.”

Algorithms for Clustering Data, Jain and Dubes

Measures of Clustering

- External validation
 - Compare clusters based on data labels
 - Similar to usual training/scoring scenario
 - Good idea if know something about data
- Internal validation
 - Determine quality based only on clusters
 - E.g., spacing between and within clusters
 - Generally applicable

Also: Relative validation

- Comparing clustering results
 - That is, compare one clustering result with others for same dataset
 - Would be very useful in practice
 - Often, lots of trial and error
 - Could enable us to “hill climb” to better clustering results...
 - ...if we have a way to quantify things

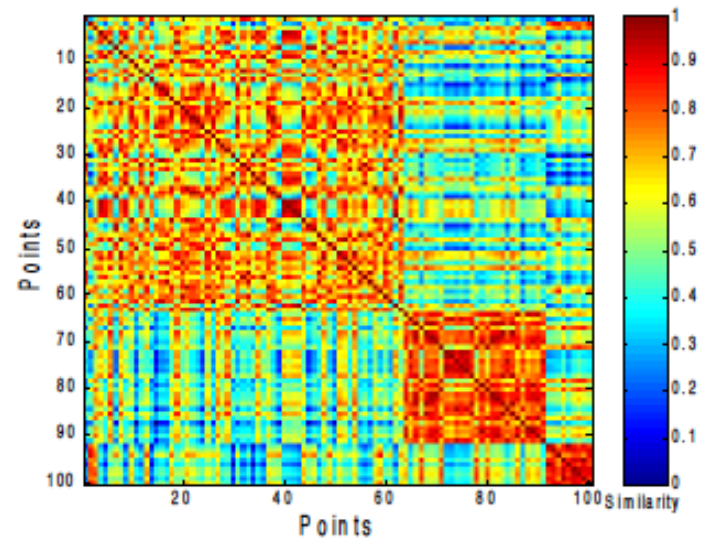
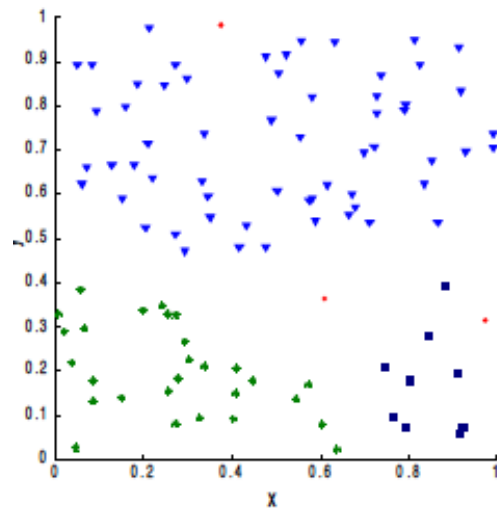
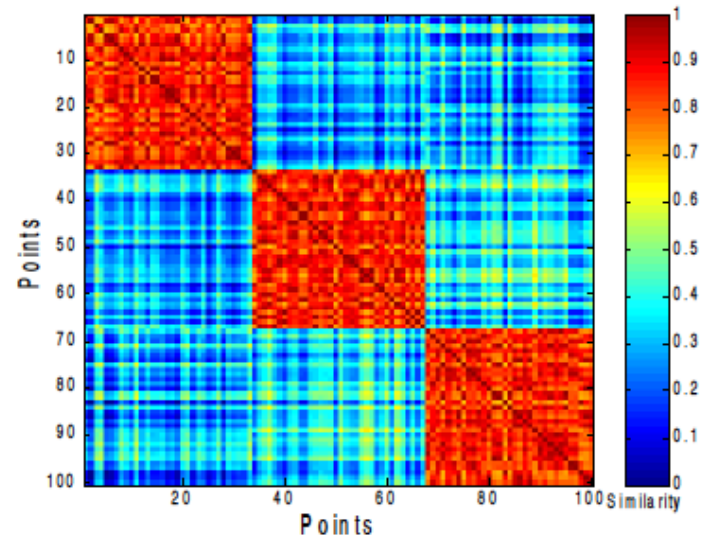
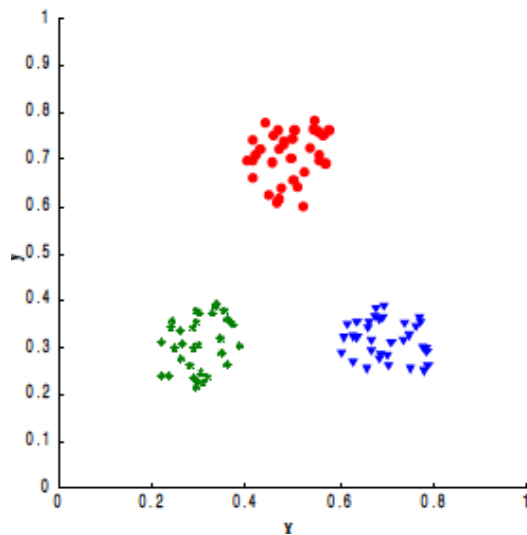
Internal Validation

- Direct measurement of clusters
- We'll consider the following
 - Cluster correlation
 - Similarity matrix
 - Sum of squares error
 - Cohesion and separation

Similarity Matrix

- Form “similarity matrix”
 - Could be based on just about anything
 - Typically, distance matrix $D = \{d_{ij}\}$, where $d_{ij} = d(x_i, x_j)$
- Group rows and columns by cluster
- Heat map for resulting matrix
 - Provides visual representation of similarity within clusters

Similarity Matrix



Cluster Correlation

- Given data $x_1, x_2, \dots, x_m$, and clusters, define 2 matrices
- Distance matrix $D = \{d_{ij}\}$
 - Where d_{ij} is distance between x_i and x_j
- Adjacency matrix $A = \{a_{ij}\}$
 - Where a_{ij} is 1 if x_i and x_j in same cluster
 - And a_{ij} is 0 otherwise
- Now what?

Cluster Correlation

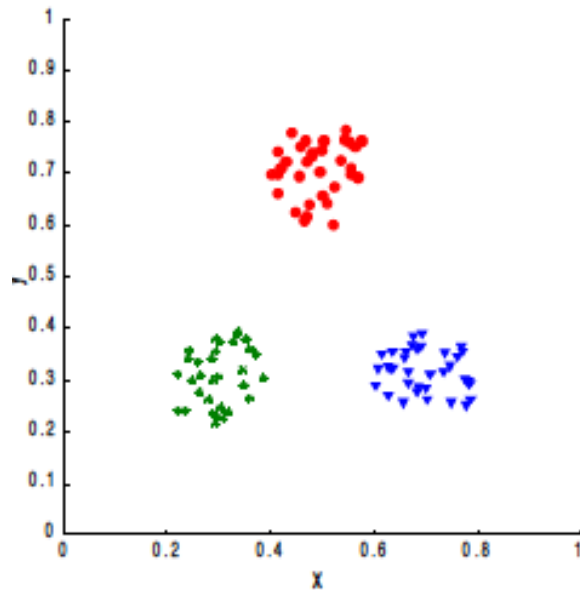
- Compute correlation between D and A

$$\begin{aligned} r_{AD} &= \text{Corr}(A,D) = \text{cov}(A,D) / (\sigma_A \sigma_D) \\ &= \Sigma(a_{ij} - \mu_A)(d_{ij} - \mu_D) / \sqrt{\Sigma(a_{ij} - \mu_A)^2 \Sigma(d_{ij} - \mu_D)^2} \end{aligned}$$

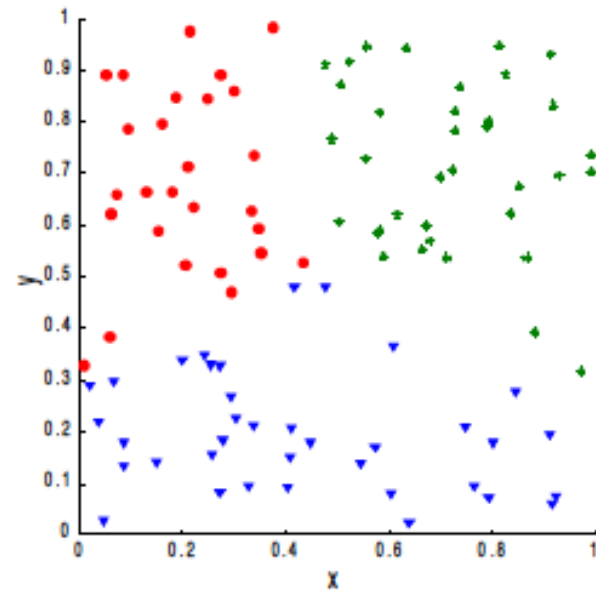
- Can show that r is between -1 and 1
 - If $r > 0$ then positive Corr (and vice versa)
 - Magnitude is strength of correlation
- High (inverse) correlation implies nearby things clustered together

Correlation

- Correlation examples



Corr = -0.9235



Corr = -0.5810

Residual Sum of Squares

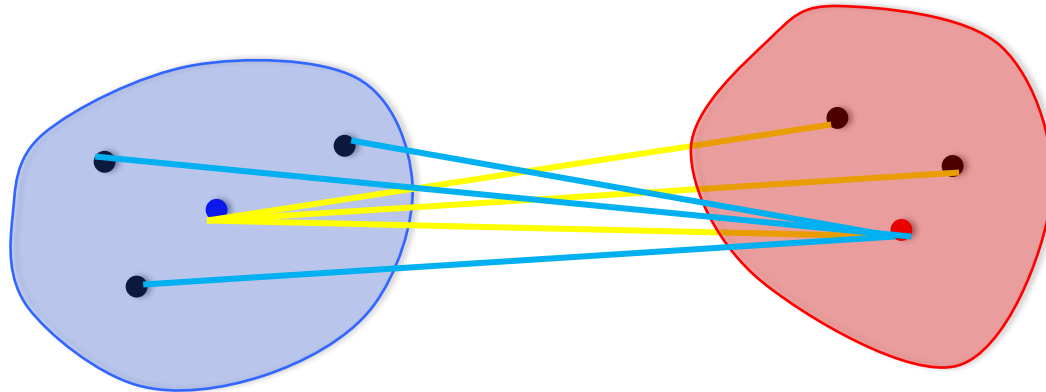
- Residual Sum of Squares (RSS)
 - Aka Sum of Squared Errors (SSE)
 - RSS is squared sum of “error” terms
 - Definition of error depends on problem
- What is “error” when clustering?
 - Distance from centroid?
 - Could use other measures instead

Cohesion and Separation

- Cluster **cohesion**
 - How tightly packed is a cluster
 - More cohesive clusters is better
- Cluster **separation**
 - Distance between clusters
 - The more separation, the better
- Easy to measure

Separation

- Many ways to measure this
 - Here, using distances to other centroids



- Or distances between all points in clusters
- Or distance between centroids, or...

External Validation

- “External” implies that we measure quality based on data in clusters
 - Not relying on cluster topology (“shape”)
- Suppose clustering data is of several different types
 - Say, different malware families

Entropy and Purity

- Entropy
 - Standard measure of uncertainty or randomness
 - High entropy implies clusters less uniform
- Purity
 - Another measure of uniformity
 - Ideally, cluster should be more “pure”, that is, more uniform

Entropy

- Suppose total of m data elements
 - As usual, $x_1, x_2, \dots, x_m$
- Denote cluster j as C_j
 - Let m_j be number of elements in C_j
 - Let m_{ij} be count of type i in cluster C_j
- Compute probabilities based on relative frequencies
 - That is, $p_{ij} = m_{ij} / m_j$

Entropy

- Then entropy of cluster C_j is
 $E_j = - \sum p_{ij} \log p_{ij}$, where sum is over i
- Compute entropy E_j for each cluster C_j
- Overall (weighted) entropy is then
 $E = \sum m_j/m E_j$, where sum is from 1 to K and K is number of clusters
- Smaller E is better
 - Implies clusters less uncertain/random

Purity

- Ideally, each cluster is all one type
- Using same notation as in entropy...
 - Purity of C_j defined as $U_j = \max p_{ij}$
 - Where max is over i (i.e., different types)
- If U_j is 1, then C_j all one type of data
 - If U_j is near 0, no dominant type
- Overall (weighted) purity is
$$U = \sum m_j/m U_j, \text{ where sum is from } 1 \text{ to } K$$

User Study

- Pick pairs of items: should they belong to the same cluster or not?